

Graduation Project

Presented at

The Higher Institute of Computer Science
And Multimedia of Sfax

in order to obtain

**BACHELOR'S DEGREE
IN COMPUTER SCIENCE:
BIG DATA AND DATA ANALYTICS**

entitled

AI-Based Skin Analysis and Lesion Detection

By

Fezzani Koussay

Presented on 03/06/2026, before the jury composed of :

Mr. Tmar Mohamed

President

Mr. Aloulou Firas

Reviewer

Mr. Zlitni Tarek

Supervisor

Academic Year : 2025-2026

DEDICATION

To my beloved parents,

Your unwavering support, sacrifices, and love have been the foundation of my journey. This achievement is a testament to your faith in me, and I dedicate it to you with profound gratitude.

To my cherished family and friends,

Your encouragement and belief in me have been my strength through every challenge.

To all who inspired me,

From my teachers at ISIMS to the professionals at Aura Dynamic who shared their insights, your guidance has shaped this work.

Koussay Fezzani

ACKNOWLEDGEMENT

I express my heartfelt gratitude to all who contributed to the success of this Final Year Project. I sincerely thank my supervisor, **Mr. Tarek Zlitni**, for his invaluable guidance and continuous support throughout the development of the DermaAI platform.

I am also grateful to the faculty of ISIMS for their academic mentorship and the knowledge they imparted during my studies. Special thanks go to my colleagues and friends for their insightful discussions and technical assistance.

Finally, I owe immense gratitude to my family for their love, patience, and unconditional support.

Koussay Fezzani

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
AKIEC	Actinic Keratoses and Intraepithelial Carcinoma
API	Application Programming Interface
BCC	Basal Cell Carcinoma
BKL	Benign Keratosis-like Lesions
CAM	Class Activation Mapping
CDN	Content Delivery Network
CNN	Convolutional Neural Network
CRUD	Create, Read, Update, Delete
DB	Database
DF	Dermatofibroma
ERP	Enterprise Resource Planning
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
LTR	Left-To-Right
MEL	Melanoma
MVC	Model-View-Controller

NLP	Natural Language Processing
NV	Melanocytic Nevi
PII	Personally Identifiable Information
RBAC	Role-Based Access Control
REST	Representational State Transfer
RTL	Right-To-Left
SPA	Single Page Application
SSE	Server-Sent Events
UI / UX	User Interface / User Experience
UML	Unified Modeling Language
VASC	Vascular Lesions
VGG	Visual Geometry Group

Contents

LIST OF ABBREVIATIONS	iv
List of Figures	x
List of Tables	xi
GENERAL INTRODUCTION	1
1 Chapter 1: General Framework Presentation and Project and Requirements	3
1.1 Introduction	4
1.2 Project Framework	4
1.2.1 Overview of the Host Organization	4
1.2.1.1 General Information	4
1.2.1.2 Services and Areas of Expertise	5
1.2.2 General Context of the Project	5
1.2.2.1 Background	6
1.2.2.2 Advantages of the Proposed Digital Approach	6
1.3 Problem Statement	7
1.3.1 Problem Statement	7
1.3.2 Existing Solutions	7
1.3.2.1 Limitations of Existing Solutions	9
1.4 Proposed Solution	9
1.5 Methodology	10
1.5.1 SCRUM Methodology	10
1.6 Requirements Specification	11
1.6.1 Functional Requirements	12
1.6.2 Non-Functional Requirements	12
1.6.3 Identification of Actors	13
1.6.4 General Use-Cases Diagram	14
1.7 Scrum Project Management	15
1.7.1 Scrum Team Identification	16
1.7.2 Project Planning	16
1.7.3 Project Backlog	16
1.8 Conclusion	18
2 Chapter 2: Technical Study	19
2.1 Introduction	20
2.2 Architectural Specifications	20

2.2.1	System Architecture	20
2.2.1.1	Comparative Study	20
2.2.1.2	Internal Architecture of the Backend System	23
2.2.2	Physical Architecture	25
2.2.3	Deployment Diagram	26
2.3	Technological Specification	27
2.3.1	Hardware Environment	27
2.3.2	Software Environment	28
2.3.2.1	Development Tools	28
2.3.2.2	Database Management Tool	28
2.3.2.3	API Management Tool	29
2.3.2.4	Project Management Tool	29
2.3.3	Technologies and Programming Languages Used	30
2.4	Conclusion	33
3	Chapter 3: Sprint 1 — Authentication & Access Management	34
3.1	Introduction	35
3.2	Task Identification	35
3.2.1	Expected Objectives	35
3.2.1.1	Modeling the Identity Management Process	35
3.2.2	Sprint Backlog	37
3.3	Functional Specifications	37
3.3.1	Use-Cases Diagram	37
3.3.2	Textual Descriptions of Use Cases	38
3.3.2.1	Authenticate	38
3.3.2.2	Consult Medical News	39
3.4	Design	39
3.4.1	Class Diagram	39
3.4.1.1	Data Dictionary	40
3.4.2	Sequence Diagrams	40
3.4.2.1	Authentication Sequence	40
3.4.2.2	Web Scraping Sequence	41
3.5	Web Scraping Implementation	42
3.5.1	Data Extraction Strategy	42
3.5.2	Caching and Performance Optimization	43
3.6	Development	43
3.6.1	User Interfaces	43
3.6.2	API Testing	46
3.7	Progress Tracking	48
3.8	Conclusion	48
4	Chapter 4: Sprint 2 — AI Scan & Diagnosis	49
4.1	Introduction	51
4.2	Objectives	51

4.3	Design	51
4.3.1	Use-Cases Modeling	51
4.3.1.1	Perform Skin Lesion Scan	52
4.3.2	Activity Diagram	52
4.3.3	Class Diagram	54
4.3.4	Sequence Diagram	54
4.4	Tasks	55
4.5	Development	55
4.5.1	User Interfaces	55
4.5.2	API Testing	57
4.6	AI Model Development	58
4.6.1	Dataset: HAM10000	58
4.6.2	Class Imbalance Strategy	60
4.6.3	Architecture Selection: Xception Model	61
4.6.4	Training Configuration & Adaptive LR Strategy	61
4.6.5	Training Progress	62
4.6.6	Evaluation on the Test Set	62
4.6.7	Model Explainability: Grad-CAM	64
4.6.8	Inference Performance	64
4.7	Progress Tracking	65
4.8	Conclusion	65
5	Chapter 5: Sprint 3 — History, AiChabot & Reports	66
5.1	Introduction	67
5.2	Objectives	67
5.3	Design	67
5.3.1	Use-Cases Modeling	67
5.3.1.1	Interact with Digital Assistant	68
5.3.2	Activity Diagram	68
5.3.3	Class Diagram	70
5.3.4	Sequence Diagram	70
5.4	Tasks	71
5.5	Development	71
5.5.1	User Interfaces	71
5.6	AI Model Development: Local Conversational Assistant	74
5.6.1	Model Selection and Inference Engine	74
5.6.2	Contextual Prompt Engineering and Localization	74
5.6.3	Real-Time Token Streaming	75
5.7	Progress Tracking	75
5.8	Conclusion	76
6	Chapter 6: Sprint 4 — Admin Dashboard & Data Export	77
6.1	Introduction	78
6.2	Objectives	78

6.3	Design	78
6.3.1	Use-Cases Modeling	78
6.3.1.1	Export System Data	79
6.3.2	Activity Diagram	79
6.3.3	Class Diagram	80
6.3.4	Sequence Diagram	80
6.4	Tasks	81
6.5	Development	82
6.5.1	User Interfaces	82
6.5.2	API Testing & Security Validation	85
6.6	Administrative Control System	85
6.6.1	Role-Based Access Control (RBAC)	86
6.6.2	Data Anonymization Pipeline	86
6.7	Progress Tracking	86
6.8	Conclusion	87
	GENERAL CONCLUSION	88
	BIBLIOGRAPHY	89

List of Figures

1.1	Aura Dynamic Company Logo	5
1.2	Home Interface of SkinVision [9]	8
1.3	Home Interface of FirstDerm [10]	8
1.4	SCRUM Process [12]	11
1.5	Global Use-Cases Diagram	15
2.1	Comparison between Monolithic and Microservices Architecture	21
2.2	Global Architecture of the Developed Solution	22
2.3	Architecture of the Backend Layer	23
2.4	Frontend Architecture of the Developed Solution	24
2.5	Physical Architecture of the Developed Solution	26
2.6	Deployment Diagram	27
2.7	Hardware Environment: Lenovo IdeaPad 3	27
2.8	GitHub Logo	28
2.9	MongoDB Compass Logo	29
2.10	Postman Logo	29
2.11	Trello Logo	29
2.12	Trello Global Workplace: Overview of Sprint Boards	30
2.13	Flask Logo	30
2.14	React Logo	31
2.15	Tailwind CSS Logo	31
2.16	Python Logo	32
2.17	TensorFlow Logo	32
2.18	NoSQL Database Logo	32
3.1	Activity Diagram for Authentication and Identity Management	36
3.2	Use-Cases Diagram for Sprint 1	38
3.3	Conceptual Class Diagram for Sprint 1	39
3.4	Sequence Diagram: Authentication	41
3.5	Sequence Diagram: Consult Medical News	42
3.6	Login and Registration Interfaces	44
3.7	User Profile Management Interface	45
3.8	Dynamic Medical News Interface	46
3.9	Postman Test: Authentication API Validation	47
3.10	Postman Test: News Scraper API Validation	47
3.11	Trello Board at the End of Sprint 1	48
4.1	Use-Cases Diagram for Sprint 2	52

4.2	<i>Activity Diagram for Scanning Process</i>	53
4.3	<i>Conceptual Class Diagram for Sprint 2</i>	54
4.4	<i>Sequence Diagram: Scanning and Prediction Flow</i>	54
4.5	<i>Interactive Body Map for Scan Location</i>	56
4.6	<i>Skin Lesion Image Upload Interface</i>	56
4.7	<i>Diagnostic Results Display</i>	57
4.8	<i>Postman Test: Deep Learning Inference API</i>	58
4.9	<i>Representative Samples for the Seven Diagnostic Classes</i>	60
4.10	<i>Training and Validation Loss & Accuracy Curves (18 Epochs)</i>	62
4.11	<i>Confusion Matrix on the 1,002-Image Test Set</i>	64
4.12	<i>Trello Board at the End of Sprint 2</i>	65
5.1	<i>Use-Cases Diagram for Sprint 3</i>	68
5.2	<i>Activity Diagram for Sprint 3</i>	69
5.3	<i>Conceptual Class Diagram for Sprint 3</i>	70
5.4	<i>Sequence Diagram: AI Chatbot Interaction</i>	70
5.5	<i>Digital Assistant Chat Interface</i>	72
5.6	<i>Scan History Management Interface</i>	73
5.7	<i>Individual Scan History Details</i>	73
5.8	<i>Example of a Generated Diagnostic PDF Report</i>	74
5.9	<i>Trello Board at the End of Sprint 3</i>	76
6.1	<i>Use-Cases Diagram for Sprint 4 (Admin Actor)</i>	79
6.2	<i>Activity Diagram for Sprint 4</i>	80
6.3	<i>Conceptual Class Diagram for Sprint 4</i>	80
6.4	<i>Sequence Diagram: Admin Dashboard Visualization</i>	81
6.5	<i>Administrative Overview Dashboard</i>	83
6.6	<i>System Health and Status Dashboard</i>	83
6.7	<i>User Directory and Role Management Interface</i>	84
6.8	<i>Database Export Configuration Interface</i>	84
6.9	<i>Dermatology News and Blog Interface</i>	85
6.10	<i>Postman Test: Admin Privilege Enforcement</i>	85
6.11	<i>Trello Board at the End of Sprint 4</i>	87

List of Tables

1.1	<i>Identified Gaps and Proposed Solutions</i>	9
1.2	<i>Comparative Table of Agile Methods</i>	10
1.3	<i>Team and Roles</i>	16
1.4	<i>Product Backlog</i>	17
2.1	<i>Differences between Monolithic and Microservices Architecture</i>	21
2.2	<i>Hardware Environment Used for Development</i>	28
3.1	<i>Sprint 1 Backlog</i>	37
3.2	<i>Textual Description: Authenticate</i>	38
3.3	<i>Textual Description: Consult Medical News</i>	39
3.4	<i>Data Dictionary for Sprint 1</i>	40
4.1	<i>Textual Description: Perform Skin Lesion Scan</i>	52
4.2	<i>Sprint 2 Backlog</i>	55
4.3	<i>HAM10000 Dataset Split Distribution</i>	58
4.4	<i>Raw Class Distribution in the Training Set</i>	59
4.5	<i>Per-Class Classification Report on the 1,002-Image Test Set</i>	63
5.1	<i>Textual Description: Interact with Digital Assistant</i>	68
5.2	<i>Sprint 3 Backlog</i>	71
6.1	<i>Textual Description: Export System Data</i>	79
6.2	<i>Sprint 4 Backlog</i>	82

GENERAL INTRODUCTION

Healthcare has been a fundamental pillar of human civilization, sustaining well-being and driving social stability across the globe. As global populations age, the demand for accessible diagnostics has spurred the adoption of advanced medical technologies, collectively known as HealthTech. Innovations such as Artificial Intelligence (AI), Deep Learning, and data analytics are revolutionizing clinical practices by enabling precision medicine and promoting early disease detection. For instance, global platforms like SkinVision have utilized AI-driven analysis to assess skin cancer risks for millions of users [2], while automated screening systems in Europe have improved diagnostic accuracy by nearly 20

In Tunisia, patients and medical practitioners face significant obstacles that limit the effectiveness of traditional dermatological care. Challenges include a critical shortage of specialized dermatologists, particularly in rural areas, high consultation costs, and long waiting times for clinical appointments [1]. Language barriers further complicate access, as many advanced diagnostic tools are available only in English or French, while a significant portion of the Tunisian population requires localized support. In regions like Sfax, individuals struggle with limited access to preliminary screening tools, resulting in delayed diagnoses for malignant lesions and increased healthcare burdens. These issues underscore the need for a localized, cost-effective, and accessible solution tailored to Tunisia's healthcare needs, particularly for the early detection of skin cancer.

To address these challenges, the **DermaAI Platform** was developed in collaboration with **Aura Dynamic**, a digital innovation agency specializing in AI-driven solutions. This report, titled **DermaAI Project Report**, documents the project's development using the **Agile Scrum** methodology. **Chapter 1** provides a comprehensive overview, covering the collaboration, global healthcare trends, project objectives,

and methodology. **Chapter 2** details the technical design, including deep learning algorithms, system architecture, and the software environment. **Chapter 3** describes the implementation of user authentication and access management. **Chapter 4** presents the core AI integration and skin lesion analysis. **Chapter 5** covers the scan history, reporting, and chatbot features, while **Chapter 6** demonstrates the administrative dashboard and system analytics. Aimed at researchers, medical practitioners, and technology developers, this work aims to advance digital dermatology and inspire accessible healthcare practices in Tunisia and beyond.

Chapter 1: General Framework of the Project and Requirements



1.1. Introduction

This chapter establishes the foundational framework for the DermaAI platform developed at Aura Dynamic. It outlines the project context, existing market limitations, and core objectives. Furthermore, it details the agile methodology, system requirements, and primary actors driving the technical implementation of the solution.

1.2. Project Framework

This project is a Graduation Project carried out within Aura Dynamic, in partial fulfillment of the requirements for the Bachelor's degree in Big Data and Data Analysis at the Higher Institute of Computer Science and Multimedia of Sfax. It involves designing and developing DermaAI, a web-based platform for AI-assisted skin lesion screening and follow-up, structured around three main actors: user, guest, and admin. This solution aims to automate preliminary skin lesion assessment and improve accessibility to dermatological screening.

1.2.1. Overview of the Host Organization

Aura Dynamic is the host company where this graduation project was carried out. Its profile, main services, and areas of expertise are outlined below.

1.2.1.1. General Information

Aura Dynamic is a Tunisian company specialized in the design and development of digital solutions, including web and mobile applications. The company focuses on transforming client ideas into reliable, scalable, and modern digital products.

Aura Dynamic supports projects across the full development lifecycle, from needs analysis and system design to development, testing, and deployment. The company operates in a dynamic and innovation-driven environment, with growing interest in integrating modern technologies such as artificial intelligence into its solutions. The company's professional profile is available on LinkedIn: <https://www.linkedin.com/company/aura-dynamic>.

The Aura Dynamic company logo is shown in Figure 1.1.



Figure 1.1: Aura Dynamic Company Logo

1.2.1.2. Services and Areas of Expertise

Aura Dynamic mainly provides the following services:

- Design and development of web applications.
- Development of mobile applications.
- UI/UX design and user experience optimization.
- Development of business management systems (ERP, CRM, etc.).
- Deployment and maintenance of digital solutions.

The company's work focuses on several key domains:

- Web development.
- Mobile development.
- Business process digitalization.
- System integration.
- Artificial intelligence (emerging focus).

1.2.2. General Context of the Project

With the rapid growth of AI applications in healthcare and the growing adoption of information technologies, medical institutions and individuals are looking for intelligent solutions to enhance access to dermatological care. Skin lesion screening, often performed manually by specialists, is time-consuming, susceptible to subjectivity, and geographically limited. This context highlights the need for an integrated and automated AI-powered platform capable of effectively supporting preliminary assessment and clinical decision-making.

1.2.2.1. Background

Skin cancer, including melanoma, represents a growing global public-health burden, with around 325,000 new cases and 57,000 deaths recorded in 2020, and incidence projected to rise through 2045 [1, 2]. Early detection significantly improves outcomes, with five-year survival rates exceeding 99% for localized melanoma versus only 35% for advanced stages [3]. Deep learning approaches have shown strong performance in skin lesion classification, marking significant progress in AI-assisted dermatology [4, 5, 6]. A dedicated intelligent screening platform provides an optimal response to these challenges by offering:

- Centralized AI-powered analysis enabling more consistent and reliable preliminary assessments.
- Near-instant screening results that support triage and referral decisions.
- Interpretable outputs providing visual explanations alongside predictions, improving clinical trust.

As a core component of this project, the DermaAI platform provides an automated workflow for uploading skin lesion images, generating AI predictions with confidence scores, producing visual explanations, and generating structured PDF reports for users.

1.2.2.2. Advantages of the Proposed Digital Approach

Developing an AI-powered screening platform provides several strategic and clinical benefits:

- **Improved Accessibility:** Users can access a preliminary screening tool without requiring an immediate specialist consultation, reducing dependence on specialist availability and lowering consultation costs.
- **Faster Preliminary Assessment:** The platform provides near-instant preliminary results, supporting triage decisions and reducing waiting times.
- **Interpretable AI Outputs:** Visual explanations highlight the analyzed region, making AI predictions more transparent and clinically useful.

1.3. Problem Statement

Skin lesion screening still faces accessibility limits, diagnostic delays, and important gaps in current digital tools. The following analysis highlights the main issues and the limits of existing solutions.

1.3.1. Problem Statement

Skin cancer is one of the most common types of cancer globally [3], yet early and accurate detection significantly increases survival rates and reduces treatment costs [7]. The traditional diagnostic process relies heavily on visual inspection performed by specialists, making results sensitive to human error and variations in clinical experience [5].

The main challenges currently faced include:

- Long waiting times before consulting a dermatologist, particularly in rural or underserved areas suffering from a shortage of specialized personnel [8].
- Unreliable self-diagnosis through internet searches or generic symptom checkers, which may cause unnecessary anxiety or dangerous delays in seeking medical advice.
- Limited access to tools capable of providing a reliable preliminary assessment to help patients decide whether urgent consultation is needed.

These constraints highlight the need for an accessible, reliable, and user-friendly platform capable of supporting preliminary skin lesion assessment and guiding patients toward appropriate medical care [6].

1.3.2. Existing Solutions

Reviewing existing platforms helps identify the strengths already available on the market and the gaps DermaAI still needs to address.

In recent years, several applications and tools have been developed to assist in skin cancer detection by combining artificial intelligence, computer vision, and mobile technology. The objective of this comparative analysis is to understand the strengths and weaknesses of existing solutions in order to better position DermaAI.

The following platforms were primarily evaluated:

“SkinVision” Platform

SkinVision [9] is a consumer-facing mobile application that uses AI to assess skin lesion images for risk levels. It offers image-based risk classification and referral advice. However, it relies on a cloud-based service and does not provide visual explanations, making it unsuitable for clinical interpretation. Figure 1.2 shows the home interface of SkinVision.

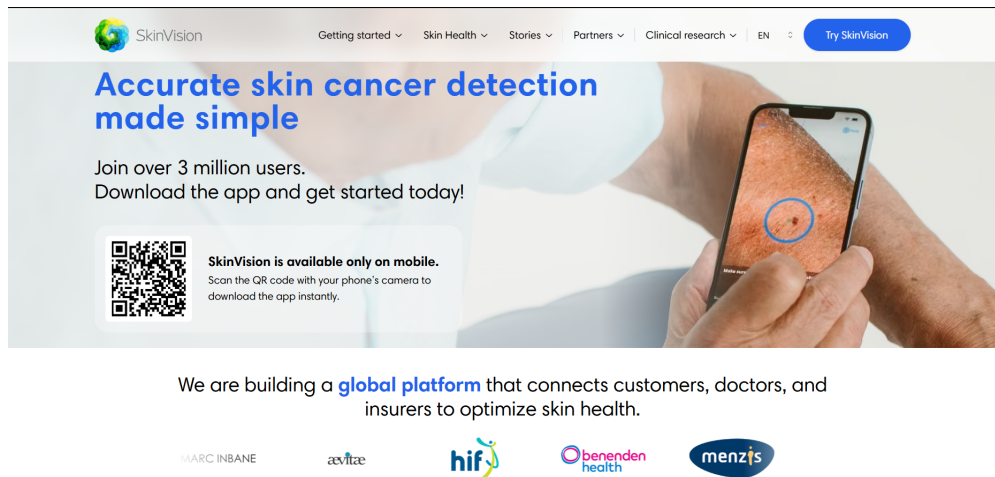


Figure 1.2: Home Interface of SkinVision [9]

“FirstDerm” Platform

FirstDerm [10] is a dermatology-oriented application that allows users to submit skin photos for assessment. It follows a pay-to-see-results model and does not provide visual explanations, which limits transparency for users. Figure 1.3 illustrates the home interface of FirstDerm.

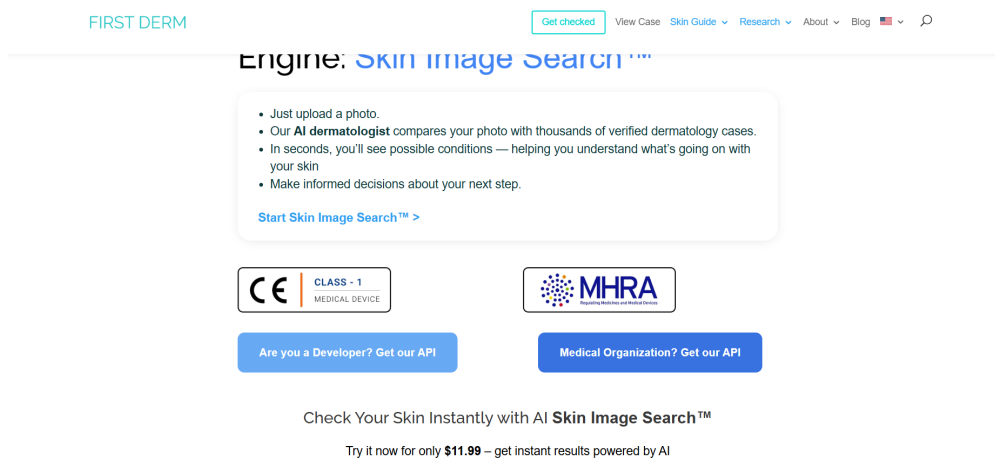


Figure 1.3: Home Interface of FirstDerm [10]

1.3.2.1. Limitations of Existing Solutions

After reviewing the platforms, the following limitations were identified:

- **Limitations of the “SkinVision” platform:**
 - No visual explanation: The platform provides a risk level without highlighting the region analyzed, limiting clinical interpretability.
 - No local deployment: The solution requires continuous cloud connectivity and cannot function in low-bandwidth environments.
 - No structured clinical reporting: Results are not formatted as clinical documents suitable for medical handover or patient records.
- **Limitations of the “FirstDerm” platform:**
 - Paid access to results: Users must pay to view the assessment, which reduces accessibility for repeated or large-scale screening.
 - No explainability: The platform does not provide visual explanations to help users better understand the result.

1.4. Proposed Solution

After a thorough analysis of existing solutions, the main objective of this project is to design and develop DermaAI, a web-based platform for AI-assisted skin lesion screening and follow-up. This solution aims to provide an accessible, explainable, and locally deployable AI platform aligned with the needs of the user, guest, and admin actors. Table 1.1 summarizes the identified gaps and the corresponding solutions retained for DermaAI.

Identified Gap	Proposed Solution
Lack of explainability in AI results	Visual and textual explanations to make screening outcomes easier to interpret.
No guidance for users after screening	Interactive AI chatbot to answer dermatology-related questions.
No structured reporting or follow-up	Comparison of scan history over time and generation of structured PDF reports.
Language barriers	Multilingual support for English, French, and Arabic.

Table 1.1: Identified Gaps and Proposed Solutions

1.5. Methodology

The development of a software project requires a rigorous methodology capable of managing complexity, ensuring quality, and respecting deadlines. Traditional linear approaches such as the Waterfall model are often too rigid for projects whose requirements evolve during implementation. This is particularly true for DermaAI, where several platform components must be refined iteratively.

For this reason, an agile approach was adopted, which promotes incremental delivery, continuous feedback, adaptability, and better risk management. During the methodological study, several agile frameworks were compared in order to identify the one that best fits the organizational and project constraints of DermaAI. The comparison is presented in Table 1.2 below.

	Advantages	Disadvantages
XP	- Pair programming - Test-driven development	- Maintenance challenges - Lack of an initial analysis phase
SCRUM	- Clear timeboxing - Strong team coordination - Incremental delivery through sprints	- Scope changes are discouraged during an active sprint
Kanban	- Very easy to implement - Continuous task flow - Visual task management	- Lacks strategic vision - Limited flexibility to meet specific client demands

Table 1.2: Comparative Table of Agile Methods

1.5.1. SCRUM Methodology

The Scrum methodology was selected for this graduation project. Scrum is an agile framework based on three pillars: transparency, inspection, and adaptation [12]. It structures development into short and manageable iterations, generally lasting two to three weeks, each ending with a potentially usable increment of the product. The roles, ceremonies, and artifacts defined by the Scrum framework provide a clear structure for incremental delivery, continuous feedback, and responsive adaptation to evolving requirements.

For DermaAI, this approach is particularly suitable because it allows the progressive integration of the main components of the platform while continuously refining prior-

ities through the Product Backlog. Figure 1.4 illustrates the standard Scrum process flow.

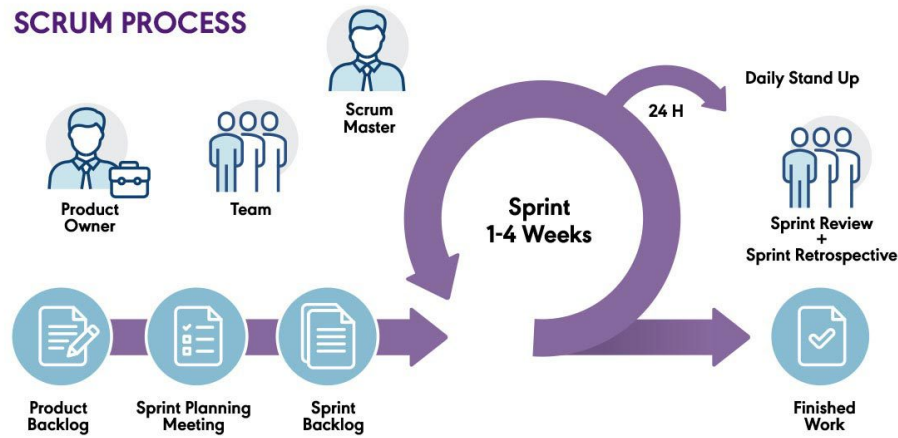


Figure 1.4: SCRUM Process [12]

As shown in Figure 1.4, the Scrum framework defines three main roles:

- **Product Owner:** In most projects, the product owner represents the client. They define the desired product features and prioritize the backlog according to project needs.
- **Scrum Master:** This role facilitates the application of the Scrum process by the team and helps maintain smooth collaboration throughout each sprint.
- **Development Team:** This team consists of engineers and students responsible for producing the product, including the author of this graduation project.

In practice, Sprint Planning, Daily Stand-ups, Sprint Reviews, and Sprint Retrospectives ensure continuous communication, regular inspection of progress, and continuous improvement throughout the project lifecycle.

1.6. Requirements Specification

DermaAI requires a clear definition of its expected features and operational constraints. The requirements retained for the platform are grouped below into functional and non-functional requirements.

1.6.1. Functional Requirements

Functional requirements describe the actions the system must be capable of performing to meet the defined objectives. The solution must provide a comprehensive set of features adapted to the needs of each identified actor. To keep the use-cases model clear and readable, similar operations are grouped under broader actions such as *manage* instead of listing each elementary action separately.

The **User** must be able to:

- Manage their personal account, profile, and preferences.
- Submit a skin lesion image together with the relevant screening context.
- Consult screening results with supporting information and visual explanations.
- Manage personal screening records, including report download, history consultation, follow-up comparison, confidence trends, and record deletion.
- Interact with the digital assistant to obtain simplified skin-health related information.

The **Guest** must be able to:

- Access the public pages of the platform.
- Manage access to the platform by creating an account or logging in securely.
- Consult educational content about skin diseases and recent dermatological news.

The **Admin** must be able to:

- Monitor overall platform activity, usage trends, system health, and logs.
- Manage registered users, roles, and access rights.
- Manage administrative data, including report export, records, and backup resources.

1.6.2. Non-Functional Requirements

Non-functional requirements represent the implicit expectations that the system must meet. They define the main quality constraints that guide the design and operation of the platform:

- **Usability / Interface Ergonomics:** The interfaces must be ergonomic, simple,

clear, and responsive. They should follow a modern clinical design style with clean layouts and readable typography on both desktop and mobile devices.

- o **Performance:** The platform must remain performant, especially for essential operations such as image upload, screening, result display, and report generation, so that users can interact with the system smoothly on both desktop and mobile devices.
- o **Security and Privacy:** Security is a critical requirement for a health-related platform. The system must ensure secure authentication, access control, and proper protection of user data.
- o **Scalability:** The platform should support a growing number of users, images, and screening records without major redesign.
- o **Maintainability:** The application must rely on a modular and well-structured architecture, with clearly separated components, in order to simplify testing, debugging, maintenance, and the integration of future improvements.
- o **Availability and Reliability:** The application must remain accessible, stable, and dependable from any location, while remaining locally deployable and limiting dependency on external cloud services.

1.6.3. Identification of Actors

DermaAI relies on three main actors, each with specific permissions and responsibilities. Their roles are expressed through grouped use cases in order to avoid unnecessary detail such as separating add, update, and delete actions when they belong to the same overall management task:

- **User (also referred to as: Registered User, End User):**
 - Submits skin lesion images and the related screening information.
 - Consults screening results, supporting information, and visual explanations.
 - Manages personal reports and screening history, including comparative follow-up data.
 - Interacts with the digital assistant.
- **Guest (also referred to as: Visitor, Unauthenticated User):**
 - Accesses public platform pages without authentication.
 - Manages access to the platform by registering or authenticating.

- Consults educational content and recent dermatological news.
- **Admin (also referred to as: System Administrator, Platform Manager):**
 - Manages users, roles, and access rights.
 - Monitors system-wide analytics, usage statistics, and platform status.
 - Manages administrative reports, records, and backup resources.

1.6.4. General Use-Cases Diagram

Based on the functional requirements defined above, a global use-cases diagram was constructed to visualize the interactions between each actor and the system. This diagram provides a high-level overview of the system's scope and serves as the primary reference for the more detailed sprint-level use-cases diagrams presented in the following chapters. Figure 1.5 presents this global use-cases diagram.

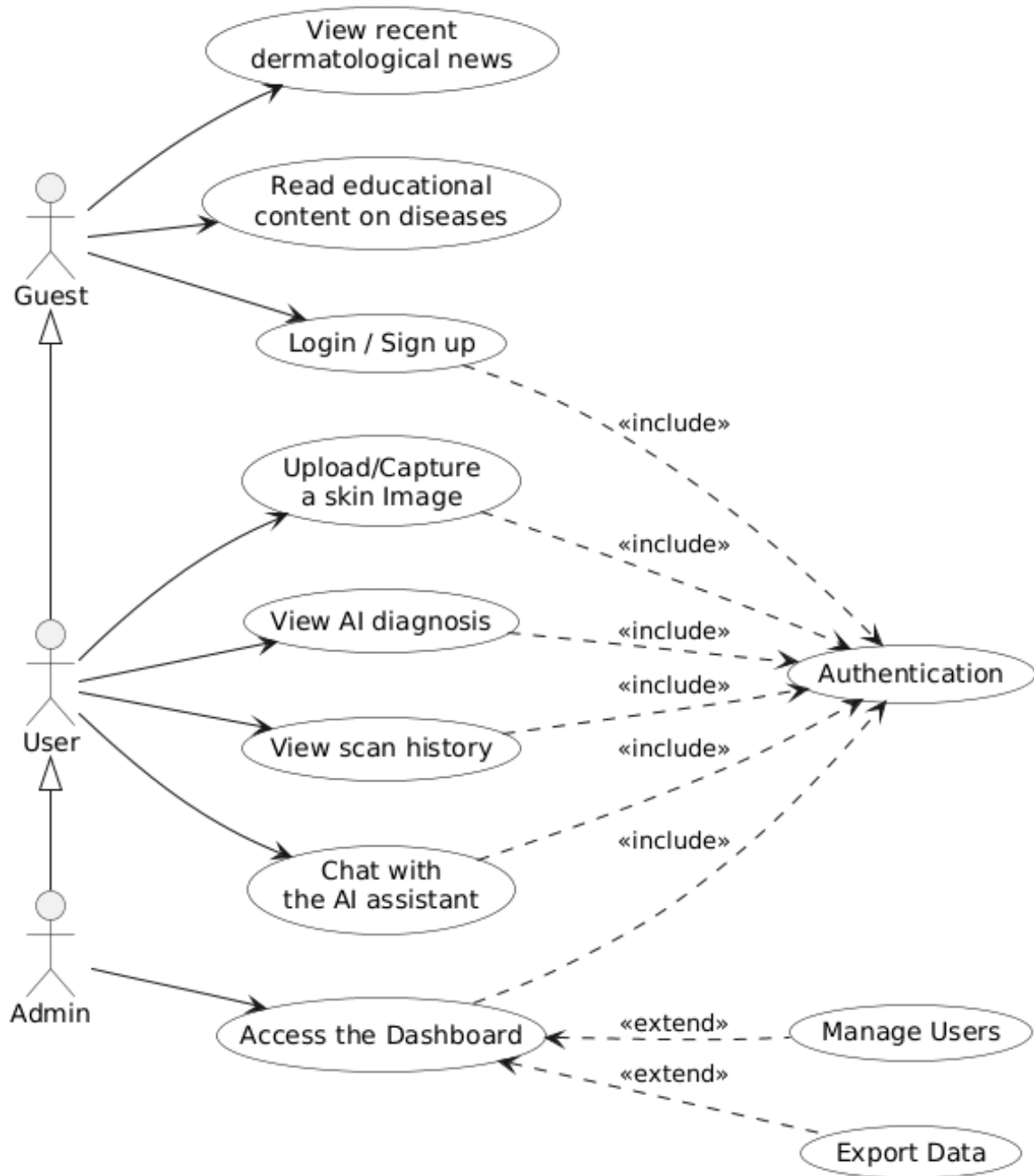


Figure 1.5: Global Use-Cases Diagram

1.7. Scrum Project Management

To ensure the successful delivery of DermaAI, the SCRUM agile framework was adopted. This approach allowed the iterative development of the main components of the platform while progressively integrating them into a coherent solution.

1.7.1. Scrum Team Identification

The project team is organized around the following operational roles, as shown in Table 1.3.

User	Scrum Roles
Mr. Omar Guetat	Scrum Master
Koussay Fezzani	Development Team

Table 1.3: Team and Roles

1.7.2. Project Planning

Dividing the graduation project into sprint-based activity batches makes its structure easier to plan, track, and communicate. Four sprints were planned in total, each addressing a distinct thematic cluster of features. The duration and content of each sprint were defined based on task complexity and interdependency.

1.7.3. Project Backlog

The product backlog groups the main features retained for DermaAI, together with their corresponding user stories and priorities, as presented in Table 1.4.

	Functionality	User Story	Priority
1	Sprint 1: Authentication & Access Management	As a guest, I want to create an account or log in securely .	High
		As a user, I want to switch the interface language (English, French, Arabic) to use the platform in my preferred language.	Medium
		As a guest, I want to read up-to-date dermatological news and articles to stay informed.	Low

2	Sprint 2: AI Scan & Diagnosis	As a user, I want to upload/Capture a skin lesion image and pinpoint its location on a body map to provide relevant screening context.	High
		As a user, I want to receive an AI-powered preliminary diagnosis with confidence scores.	High
		As a user, I want to view a visual heatmap over my image to understand exactly which areas the AI analyzed.	High
3	Sprint 3: History, AiChatbot & Reports	As a user, I want to interact with a smart digital assistant to ask follow-up questions about my screening results.	Medium
		As a user, I want to generate and download a comprehensive PDF report of my scan for my medical records.	Medium
		As a user, I want to view my scan history and delete my personal records if needed.	Medium
4	Sprint 4: Admin Dashboard & Data Export	As an admin, I want a centralized dashboard to monitor system activity, usage trends, and platform health.	Medium
		As an admin, I want to manage registered users	Medium
		As an admin, I want to export database for backup and dataset for future training .	Low

Table 1.4: Product Backlog

1.8. Conclusion

In this chapter, the professional context and requirements of DermaAI were defined, justifying the choice of the SCRUM methodology. The established functional specifications and project planning provide a clear roadmap for the upcoming development. The next chapter will present the proposed architecture and technical choices retained for the system.

Chapter 2: Technical Study



2.1. Introduction

This chapter presents the logical and physical architecture of the DermaAI web application, justified through a comparative analysis of software design patterns. It details the technologies selected for the frontend, backend, database, and deep learning components. Finally, the hardware and software development environment used for implementation is documented.

2.2. Architectural Specifications

The following sections outline and justify the logical and physical architectures selected for DermaAI.

2.2.1. System Architecture

The logical architecture describes how the different components of the application interact with each other. A comparison of different architectural styles is presented below before justifying the selection of the most suitable one for the application.

2.2.1.1. Comparative Study

Software architecture defines the structure of software components and their relationships. Two major software architectures are commonly used:

- **Microservices Architecture:** It divides an application into small, independent services that can be developed, deployed, and scaled separately.
- **Monolithic Architecture:** It organizes the application as a single unified system in which the business logic, data access, and user interface are packaged together.

Figure 2.1 provides a visual comparison between the two architectural styles.

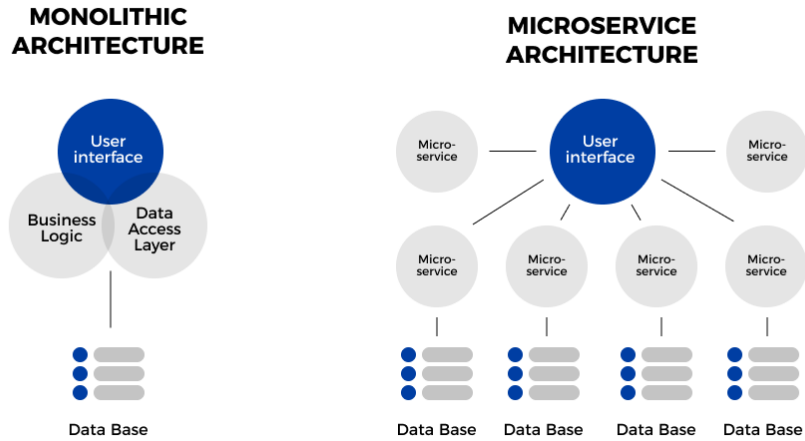


Figure 2.1: Comparison between Monolithic and Microservices Architecture

To better understand the differences between these two architectural models, Table 2.1 compares their main characteristics, strengths, and weaknesses.

Aspect	Monolithic Architecture	Microservices Architecture
Development	Easy to develop since everything is within a single codebase.	More difficult due to distributed services.
Testing	Easy to test as a monolithic unit.	Requires testing multiple independent services.
Coupling	All components are tightly coupled.	Services are loosely coupled and independent.
Technology Stack	Limited flexibility; a single tech stack is used.	Each microservice can use different technologies.
Scalability	Difficult to scale individual components independently.	Can scale individual services based on demand.
Reliability	Failure in one module can crash the entire application.	Failure in one service does not affect others.
Deployment	Requires redeployment of the entire application to update.	Can be deployed separately.
Maintenance	Becomes harder to maintain as the application grows.	Easier to update and maintain specific services.

Table 2.1: Differences between Monolithic and Microservices Architecture

After reviewing the criteria for both architectures, a **Modular Monolithic Architecture** was adopted for the implementation of DermaAI. Rather than fragmenting the application into separate, distributed microservices—which introduces unnecessary deployment and orchestration overhead for a project of this scale—a modular mono-

lith keeps all backend components within a single codebase while maintaining logical separation.

This choice is justified by several reasons:

- **Development Speed & Simplicity:** A single backend application simplifies the development environment, testing, and debugging.
- **Logical Separation:** Modular routing enforces strict boundaries between different services (e.g., authentication, deep learning inference, analytics) without the complexity of network calls between services.
- **Native Deep Learning Integration:** Keeping the AI inference pipeline within the same memory space as the API avoids heavy data serialization overhead when passing high-resolution images.
- **Ease of Deployment:** A modular monolith requires only a single deployment pipeline, making it easier to maintain and host.

The system is organized into several modular components and services, as illustrated in Figure 2.2.

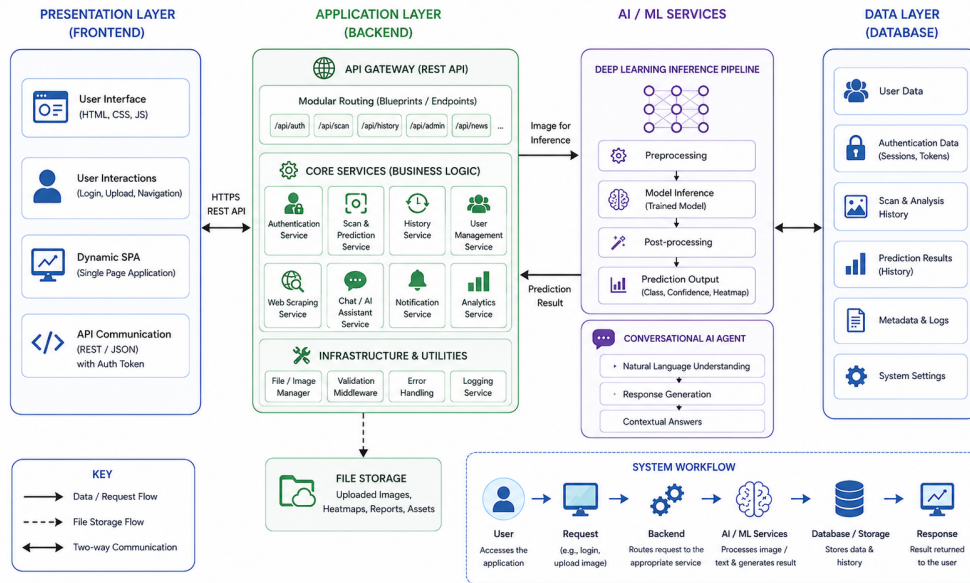


Figure 2.2: Global Architecture of the Developed Solution

System Overview:

The system works as follows:

- A user initiates an HTTP request through the frontend client.

- The request is processed and directed to the Core Backend REST API.
- The REST API routes the request to the appropriate internal module (e.g., Auth, Deep Learning Service, Web Scraper) via modular routing.
- Once processing is complete, the result is formatted as JSON and returned to the user through an HTTP response.

2.2.1.2. Internal Architecture of the Backend System

To remain efficient and maintainable, the system follows a strict Client-Server architecture with a clearly decoupled frontend and backend.

Backend Architecture

Figure 2.3 illustrates the internal structure of the backend layer.

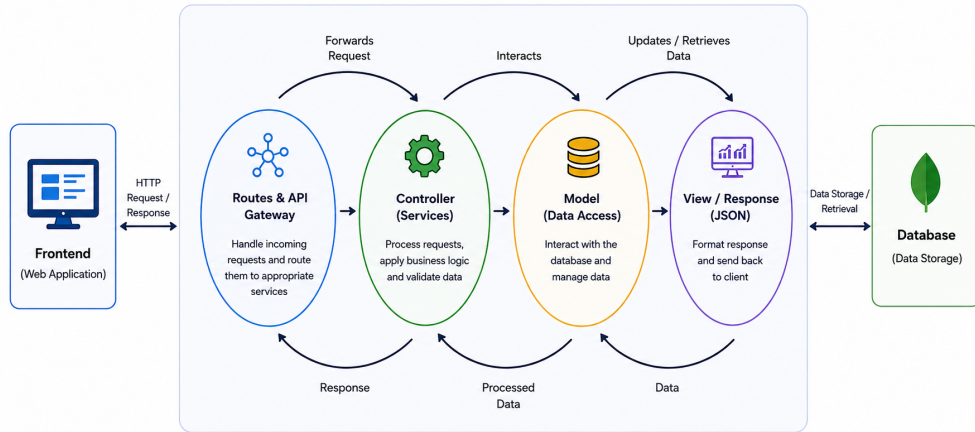


Figure 2.3: Architecture of the Backend Layer

The backend of the application is implemented using an API-driven modular architecture. Instead of traditional MVC where the backend renders views, this architecture strictly separates the data processing layer from the presentation layer.

Key Backend Components:

- **Data Layer & Repositories:** This handles database interactions using a NoSQL database driver. It is responsible for reading and writing data, validating entities prior to persistence, and defining data structures for users, scan histories, and system logs.
- **Routing System:** Modular routing mechanisms are used to logically organize the API endpoints. Related endpoints are grouped into dedicated modules such

as `/api/auth`, `/api/scan`, `/api/history`, `/api/admin`, and `/api/news`. This ensures the codebase remains modular and scalable.

- **Service Logic:** Business rules and complex operations are extracted from the routes and placed in dedicated services. This includes the `prediction_service` (managing the deep learning inference and heatmap generation), the `chat_service` (managing the local AI conversational model), and the **Web Scraping Service** (using web scraping tools to actively extract real-time news).

AI Component

The artificial intelligence layer is seamlessly integrated within the backend architecture. It consists of two main pillars:

- **Deep Learning Inference Pipeline:** A convolutional neural network (CNN) model handles the primary task of skin lesion classification. It processes user-uploaded images, extracts features, and outputs a diagnostic probability along with a visual heatmap to explain the prediction.
- **Conversational AI Agent:** A natural language processing model serves as an intelligent chatbot, providing contextual information and answering dermatological queries to support the users.

Frontend Architecture

Figure 2.4 shows the component-based structure of the frontend framework.

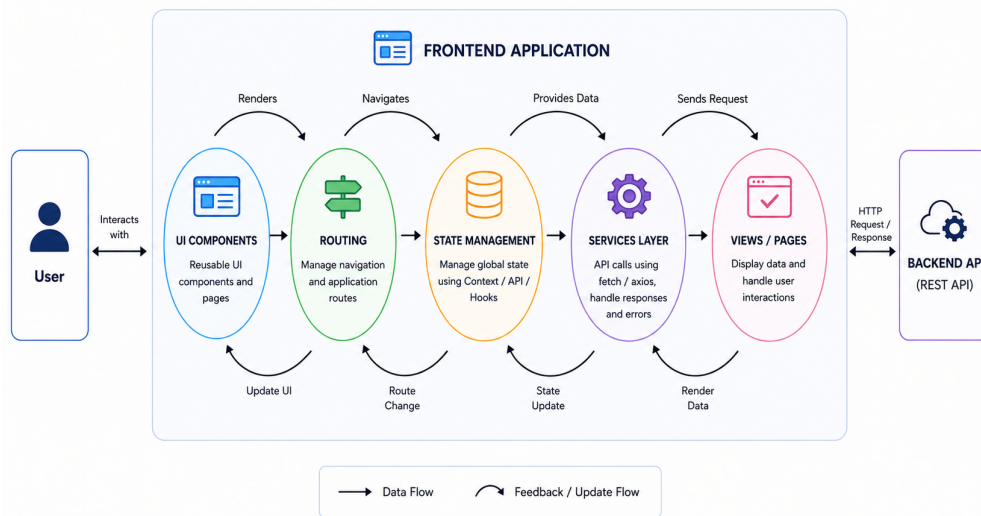


Figure 2.4: Frontend Architecture of the Developed Solution

The frontend of the application is developed using a modern JavaScript library and follows a component-based architecture. The interface is organized into reusable components, contexts, and specific pages, which improves modularity and simplifies maintenance.

Key Frontend Components:

- **Modules / Contexts:** State management Contexts and modules are logical repositories that collect and organize related features such as global state, themes, authentication sessions, and localization contexts across the application.
- **Components:** All frontend applications consist of one or more components, which are the fundamental building blocks of the user interface. Components are JavaScript functions that return template syntax (e.g., Navbar, interactive BodyMap, ResultsDisplay).
- **Templates:** The frontend uses a syntax extension for JavaScript that combines markup and component logic, making it easier to describe dynamic interfaces.
- **Routing System:** A client-side routing system is used for handling navigation among the different views of the application without requiring full page reloads.
- **HTTP Fetch API:** The frontend relies natively on the JavaScript Fetch API to interact with the backend. Dedicated service functions attach authorization headers (authorization tokens) and handle API request lifecycles transparently.

2.2.2. Physical Architecture

The choice of physical architecture is an important decision as it directly affects the performance and scalability of the system. A three-tier architecture was adopted, which extends the classic client-server model.

Presentation Layer

This layer corresponds to the user interface (UI), built with modern frontend technologies. Its primary role is to consume the backend API and present the platform's services in a seamless, responsive layout on any device.

Business Logic Layer

This layer handles the core operations of the application. It processes requests from the presentation layer and executes business rules. It hosts the RESTful API, the deep learning inference pipeline, the local AI chatbot model, and the data extraction scripts.

Data Access Layer

This layer manages interaction with the NoSQL database for data retrieval and storage, preserving data integrity for user profiles, scan histories, heatmap paths, and system logs.

Figure 2.5 presents the three-tier physical architecture of the system.

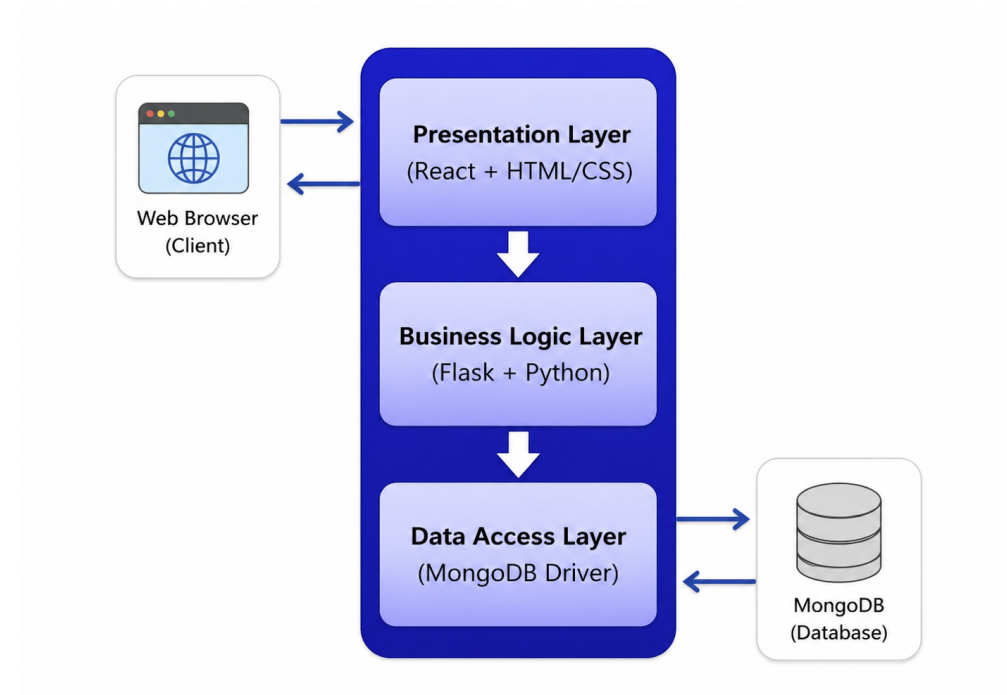


Figure 2.5: Physical Architecture of the Developed Solution

2.2.3. Deployment Diagram

To model the physical architecture of the application and illustrate the relationships between software components and hardware infrastructure, the deployment diagram is presented below in Figure 2.6.

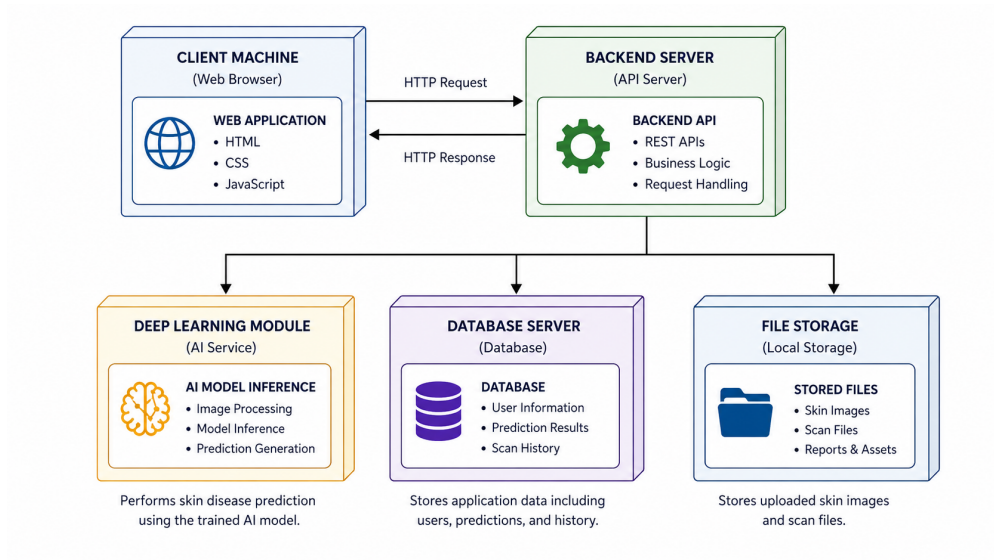


Figure 2.6: Deployment Diagram

2.3. Technological Specification

This section presents the hardware and software environment used during application development.

2.3.1. Hardware Environment

The NVIDIA GeForce RTX 3050 Ti Laptop GPU, equipped with 2560 CUDA cores and 4 GB of dedicated GDDR6 memory, was leveraged to accelerate the deep learning training and inference pipelines using CUDA-enabled TensorFlow.



Figure 2.7: Hardware Environment: Lenovo IdeaPad 3

PC	Portable
RAM	16 GB
Processor	AMD Ryzen 5 5600H
Storage	476.9 GB SSD
GPU	NVIDIA GeForce RTX 3050 Ti Laptop (4 GB GDDR6)
Operating System	Windows 11

Table 2.2: Hardware Environment Used for Development

2.3.2. Software Environment

In this section, we will present and detail the various software tools used in our work.

2.3.2.1. Development Tools

Visual Studio Code (VS Code) was utilized as the primary text editor for frontend and backend development. Its support for multiple languages and libraries, alongside useful extensions, facilitated efficient code writing for Python, React, HTML5, and CSS3.

GitHub was used for version control, collaboration, and source-code management throughout the project. It is designed to coordinate the work of programmers and track changes in any set of files.



Figure 2.8: GitHub Logo

2.3.2.2. Database Management Tool

MongoDB Compass is the graphical interface used to interact with the database during development. It allows us to explore data, run queries, inspect collections, and visualize schemas more efficiently.



Figure 2.9: MongoDB Compass Logo

2.3.2.3. API Management Tool

Postman is a popular API client that allows developers to easily create, share, test, and document APIs. It enables users to create and save simple and complex HTTP/s requests, as well as read their responses.



Figure 2.10: Postman Logo

2.3.2.4. Project Management Tool

Trello is a popular project management tool that uses boards, lists, and cards to help organize tasks and projects. It offers an intuitive interface for tracking progress, assigning tasks, and collaborating with team members. Trello is highly visual and flexible, making it an ideal choice for agile teams to manage Scrum sprints.



Figure 2.11: Trello Logo

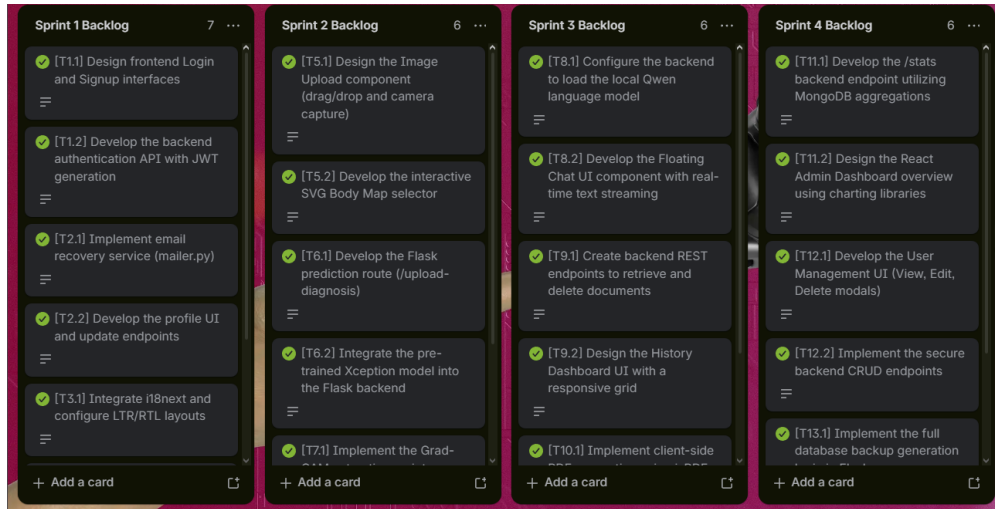


Figure 2.12: Trello Global Workplace: Overview of Sprint Boards

2.3.3. Technologies and Programming Languages Used

In this section, we will get introduced to the technologies and programming languages employed to develop our application.

- **Flask** is an open-source Python micro-framework for developing modern web applications and REST APIs. In our project, it serves as the core backend framework and supports a modular organization of the application's business logic, routing, and data management.



Figure 2.13: Flask Logo

Flask Backend Selection

- o **Modular Architecture:** It offers a clear separation of concerns via Blueprints, making the app organized, maintainable, and scalable.
- o **Native Deep Learning Integration:** Python allows direct integration with TensorFlow and Keras models, enabling seamless execution of the Xception inference pipeline.
- o **Rapid Development:** Simple routing and a lightweight nature make development fast and straightforward.

- o **Testing and Debugging:** Flask supports built-in testing features for unit and integration testing, which ensures high-quality code.
- **React** is a widely used open-source JavaScript library for building modern web applications using JavaScript and JSX. It enables the development of Single Page Applications (SPA), where content dynamically changes based on the context without reloading the entire page. React follows a component-based architecture and provides a rich set of tools for building highly performant and scalable web applications.



Figure 2.14: React Logo

React Frontend Selection

- o **Component-Based Architecture:** React has a modular architecture, which allows for simple organization, maintenance, and scaling of applications.
- o **JavaScript / JSX:** React is built with JavaScript and JSX, which improve code quality and development productivity by writing UI elements directly in JavaScript.
- o **Virtual DOM:** React's Virtual DOM ensures fast and efficient updates to the user interface, which makes UI updates simple and performant.
- o **Hooks & Contexts:** React allows for the creation of custom hooks and contexts, so application state management can be made flexible and reusable.
- **Tailwind CSS** is a utility-first CSS framework that builds on CSS. It provides support for predefined classes and responsive design utilities, and is more flexible and efficient compared to traditional CSS for rapid UI development.



Figure 2.15: Tailwind CSS Logo

- **Python** is a widely used open-source high-level programming language for web de-

velopment and artificial intelligence. It is used to create dynamic backend systems, execute deep learning inferences using TensorFlow and Keras, and run standalone scripts.



Figure 2.16: Python Logo

- **TensorFlow / Keras** is an open-source deep learning framework used to build, train, and deploy the core model of the DermaAI platform. It supports the Xception architecture and provides all the tools required for transfer learning, fine-tuning, and GPU-accelerated inference.



Figure 2.17: TensorFlow Logo

- **NoSQL Database (MongoDB)** is a popular NoSQL database management system. It is widely used for web applications and known for its speed, reliability, flexible schema, and ease of use. MongoDB stores data in JSON-like BSON documents, perfectly aligning with JavaScript frameworks and our API response structures.



Figure 2.18: NoSQL Database Logo

NoSQL Database Selection

- o Easy to configure and use.
- o NoSQL databases like MongoDB have extensive documentation and an active community.
- o NoSQL DB delivers fast performance, especially for simple queries, and scales horizontally, making it highly suitable for storing growing volumes of medical scan histories and diagnostic records.

- **BeautifulSoup4 & PyMongo & JWT:** In addition to the core frameworks, Python utility libraries such as PyMongo (for database communication), BeautifulSoup (for web scraping), and Flask-JWT-Extended (for security) were crucial to developing the full backend infrastructure.

2.4. Conclusion

This chapter presented the architectural choices behind DermaAI, justifying the selection of a Modular Monolithic approach with a decoupled React frontend. It also detailed the hardware and deep learning tools, such as TensorFlow and Keras, required for implementation. The next chapter introduces the first sprint, focusing on access management and identity features.

Chapter 3: Sprint 1 — Authentication & Access Management



3.1. Introduction

This chapter details the realization of the first development sprint, focusing on establishing the platform's foundational identity and access layer. It covers the implementation of a secure JWT-based authentication system, user profile management, and interface localization. Additionally, the development of a dynamic medical news feed via web scraping is presented.

3.2. Task Identification

This section identifies the specific expected objectives for this sprint and breaks them down into actionable development tasks.

3.2.1. Expected Objectives

During this sprint, the following core objectives were established:

- Implement a secure JWT-based authentication system (Login, Registration, Password Recovery).
- Develop the user profile management interface.
- Integrate a global language switcher supporting English, French, and Arabic.
- Develop a web-scraping service to dynamically fetch dermatological news for guests.

3.2.1.1. Modeling the Identity Management Process

To illustrate the logical flow of user authentication, Figure 3.1 presents the activity diagram modeling the login and password recovery process.

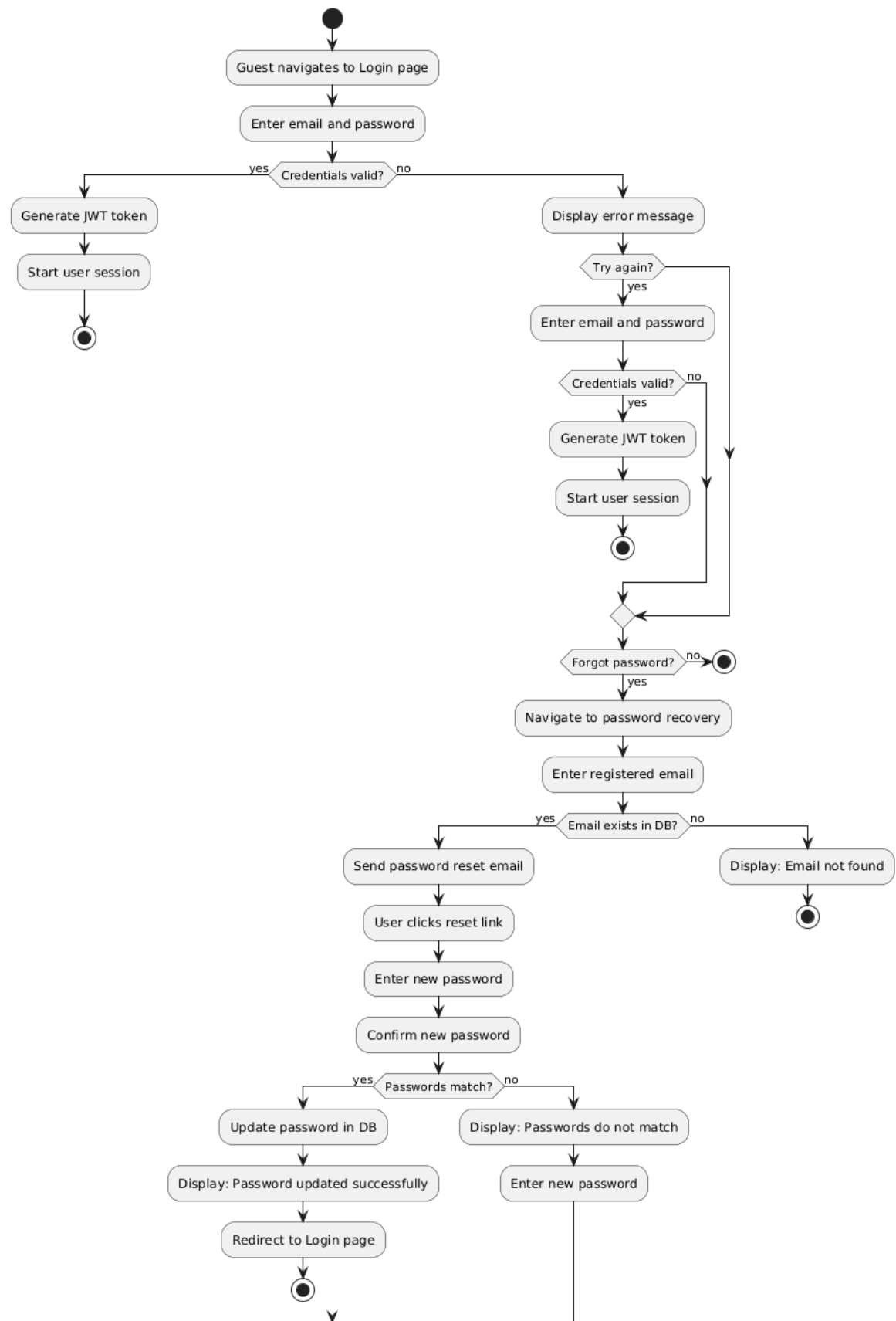


Figure 3.1: Activity Diagram for Authentication and Identity Management

3.2.2. Sprint Backlog

The tasks associated with Sprint 1 are documented in Table 3.1.

ID	User Story	Task Description
1	As a guest, I want to create an account or log in securely.	Design frontend Login and Signup interfaces.
		Develop the backend authentication API with JWT generation.
2	As a user, I want to recover my password and update my profile.	Implement email recovery service (mailer.py).
		Develop the profile UI and update endpoints.
3	As a user, I want to switch the interface language.	Integrate i18next and configure LTR/RTL layouts.
4	As a guest, I want to read up-to-date dermatological news.	Develop BeautifulSoup scraper for medical news sources.
		Create the Blog UI to display fetched articles.

Table 3.1: Sprint 1 Backlog

3.3. Functional Specifications

To ensure a clear understanding of the system's requirements for this sprint, UML Use-Cases diagrams were utilized, accompanied by detailed textual descriptions.

3.3.1. Use-Cases Diagram

Figure 3.2 illustrates the specific use cases for Sprint 1, involving both the Guest and User actors.

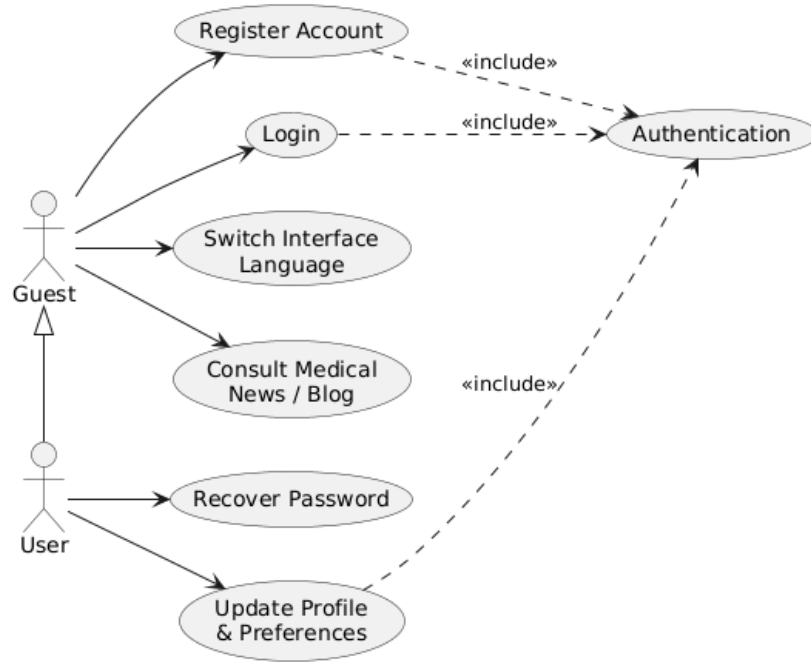


Figure 3.2: Use-Cases Diagram for Sprint 1

3.3.2. Textual Descriptions of Use Cases

3.3.2.1. Authenticate

Use Case	Authenticate
Actor	Guest
Precondition	The guest is not logged in.
Postcondition	The guest is authenticated and redirected to the application.
Nominal Scenario	1. The guest enters their email and password. 2. The system verifies the credentials. 3. A JWT token is generated and returned. 4. The user session starts.
Alternative	If credentials are invalid, the system displays an error message and denies access.

Table 3.2: Textual Description: Authenticate

3.3.2.2. Consult Medical News

Use Case	Consult Medical News
Actor	Guest, User
Precondition	The platform is connected to the internet.
Postcondition	The latest dermatological news is displayed.
Nominal Scenario	1. The user navigates to the Blog section. 2. The backend dynamically scrapes active medical sources. 3. The articles are parsed, cached, and displayed to the user.
Alternative	If the external news sites are unreachable, the system displays an empty state or a network error message.

Table 3.3: Textual Description: Consult Medical News

3.4. Design

This section details the static structure and dynamic behavior of the system elements implemented in this sprint.

3.4.1. Class Diagram

Figure 3.3 shows the conceptual class diagram for the entities managed during this sprint. Note that News Articles are dynamic and not stored persistently, unlike User profiles.

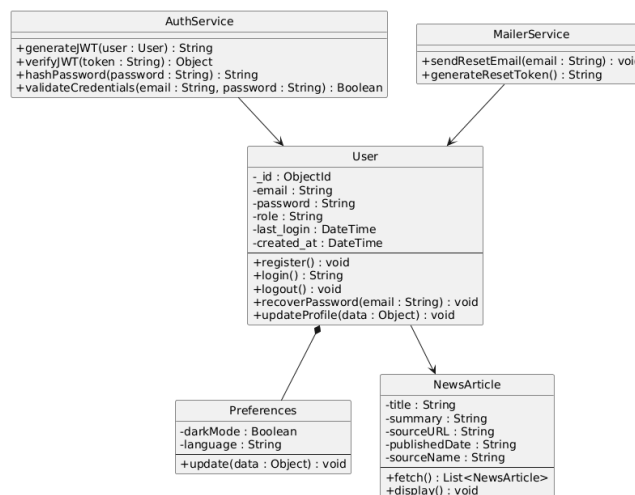


Figure 3.3: Conceptual Class Diagram for Sprint 1

3.4.1.1. Data Dictionary

Table 3.4 details the internal properties of the MongoDB `users` collection, governed by the Pydantic schemas.

Entity	Attribute	Description
User	<code>_id</code>	Unique Object Identifier generated by MongoDB.
	<code>email</code>	The user's email address used for authentication.
	<code>password</code>	The bcrypt-hashed password string.
	<code>role</code>	Access level indicator (e.g., 'user', 'admin').
	<code>preferences</code>	Object containing UI preferences (darkMode, language).
	<code>last_login</code>	Timestamp of the user's most recent session.
	<code>created_at</code>	Timestamp indicating when the account was registered.

Table 3.4: Data Dictionary for Sprint 1

3.4.2. Sequence Diagrams

3.4.2.1. Authentication Sequence

Figure 3.4 details the interactions between the React frontend, the Flask API, and the MongoDB database during a login attempt.

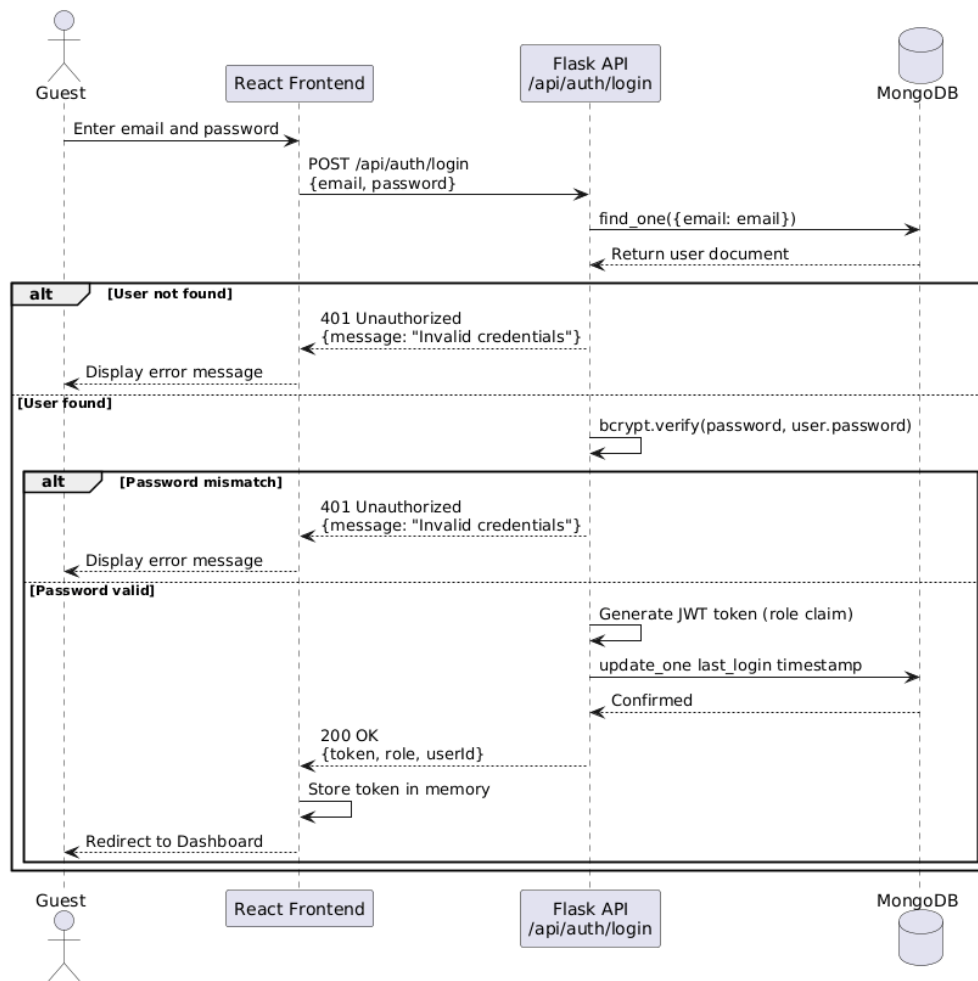


Figure 3.4: Sequence Diagram: Authentication

3.4.2.2. Web Scraping Sequence

Figure 3.5 illustrates how the backend dynamically fetches and caches news from external sources when requested by the user.

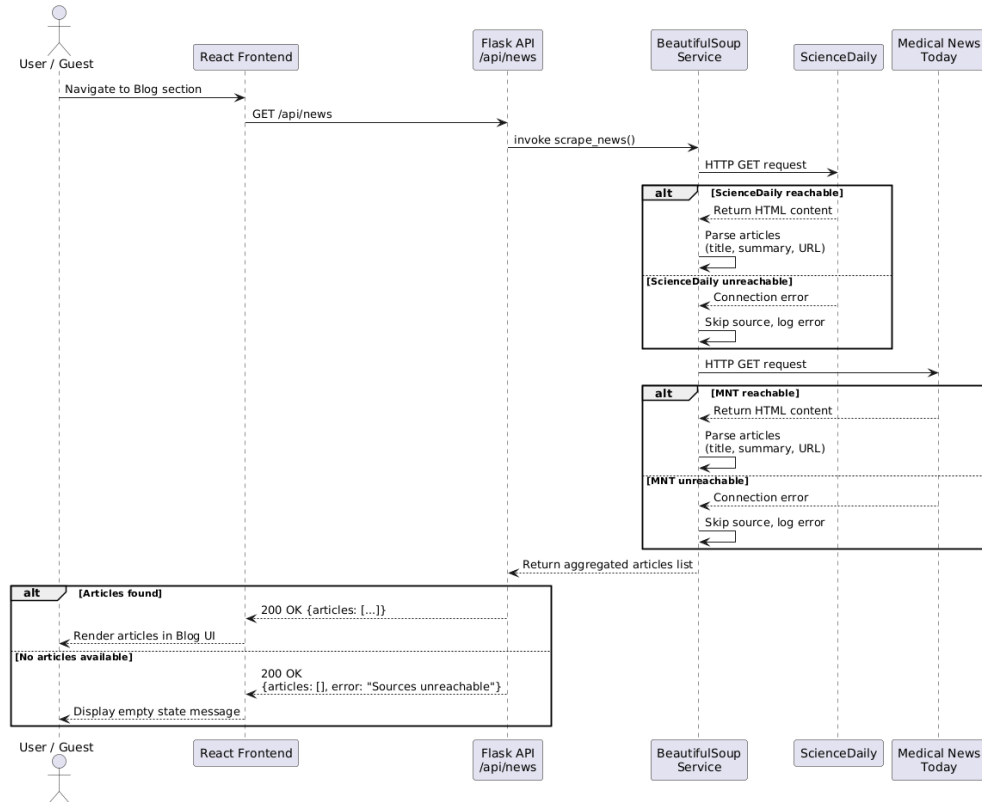


Figure 3.5: Sequence Diagram: Consult Medical News

3.5. Web Scraping Implementation

To keep users informed about the latest advancements in dermatology and skin cancer research, a dynamic web scraping service was developed using Python’s `BeautifulSoup4` and `requests` libraries. Instead of relying on static content or paid APIs, the platform actively extracts real-time articles from two highly reputable medical sources: *ScienceDaily* and *Medical News Today*.

3.5.1. Data Extraction Strategy

The extraction logic is encapsulated within dedicated functions for each source. Each function sends an HTTP GET request to the target URLs using custom **User-Agent** headers to avoid being blocked by anti-bot mechanisms. The HTML DOM is then parsed using `html.parser`.

For *ScienceDaily*, the script targets specific `div` classes (`latest-head` and `latest-summary`). For *Medical News Today*, which uses a dynamic class structure, the script relies on CSS selectors targeting specific list items containing article links. For both sources, the system successfully extracts the article’s title, publication date,

a brief summary, the source URL, and an associated thumbnail image (if available).

3.5.2. Caching and Performance Optimization

Web scraping is computationally expensive and depends heavily on external network speeds. Running the scraping logic on every user request would result in severe performance bottlenecks and risk triggering IP bans from the target websites.

To mitigate this, a global caching mechanism was implemented. The backend maintains an in-memory dictionary (`_news_cache`) containing the fetched articles and a `last_fetched` timestamp. A **Time-To-Live (TTL)** of 3600 seconds (1 hour) was configured. When the `/api/news` endpoint is called:

1. The system checks if the cache is empty or if the TTL has expired.
2. If expired, the scraper fetches fresh articles from both sources, shuffles them randomly to present a unified feed, updates the cache, and records the new timestamp.
3. If valid, the system instantly returns the cached data.

Finally, the endpoint implements server-side pagination, accepting `page` and `limit` parameters. This ensures the frontend only loads a specific number of articles at a time, guaranteeing optimal rendering performance on the client side.

3.6. Development

This section presents the final deliverables of the sprint, showcasing the functional user interfaces and validating the backend logic via API testing.

3.6.1. User Interfaces

Login and Registration Interfaces

The authentication interfaces were designed to be clean and intuitive, following modern aesthetic guidelines. The forms validate user input natively before transmitting data to the server. Figure 3.6 presents the login and registration screens.

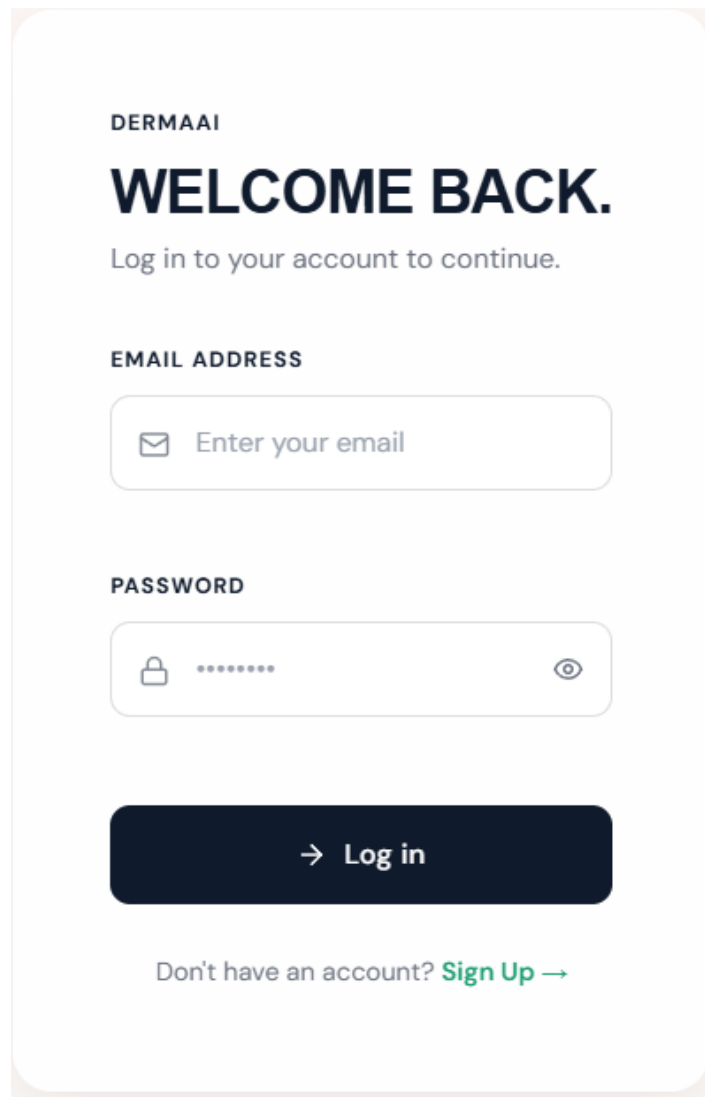
The image shows a mobile app login and registration screen. At the top, the text "DERMAAI" is displayed in a small, dark font. Below it, the heading "WELCOME BACK." is in a large, bold, dark font. Underneath the heading, the text "Log in to your account to continue." is in a smaller, gray font. The form has two main sections: "EMAIL ADDRESS" and "PASSWORD". The "EMAIL ADDRESS" section features a rounded rectangular input field with a light gray border. Inside the field, on the left, is a small envelope icon, and to its right is the placeholder text "Enter your email". The "PASSWORD" section also has a rounded rectangular input field with a light gray border. Inside, on the left, is a small padlock icon, followed by seven dots representing the password. On the right side of the password field is a small eye icon. Below these two input fields is a large, dark blue button with rounded corners. The button contains the text "→ Log in" in a white font. At the bottom of the screen, there is a line of text: "Don't have an account? Sign Up →", where "Sign Up" is in a green font and "→" is in a dark gray font.

Figure 3.6: Login and Registration Interfaces

Profile Management Interface

The profile interface enables users to view their account details, change passwords, and manage preferences, maintaining visual consistency with the platform. Figure 3.7 shows this interface.

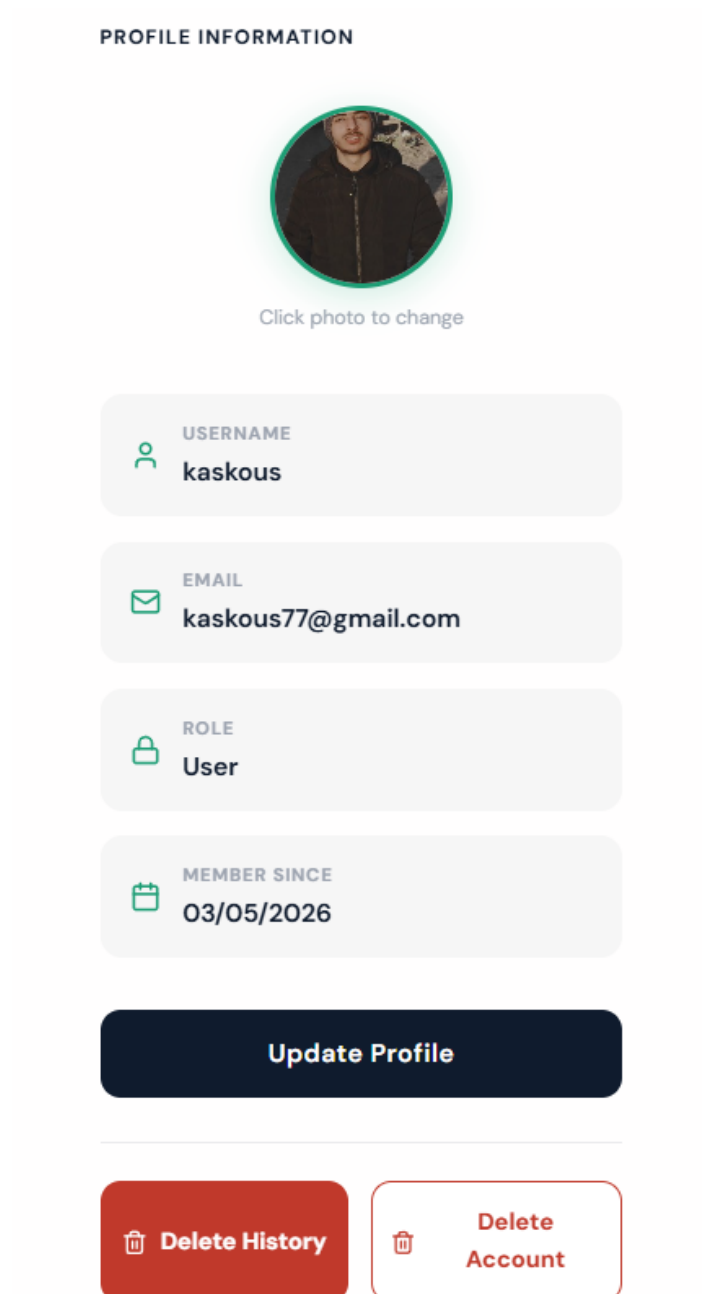


Figure 3.7: User Profile Management Interface

Medical News (Blog) Interface

The Blog page displays the dynamically scraped articles in a responsive grid layout. The interface includes pagination and language support, allowing guests to stay informed. Figure 3.8 shows the medical news interface.

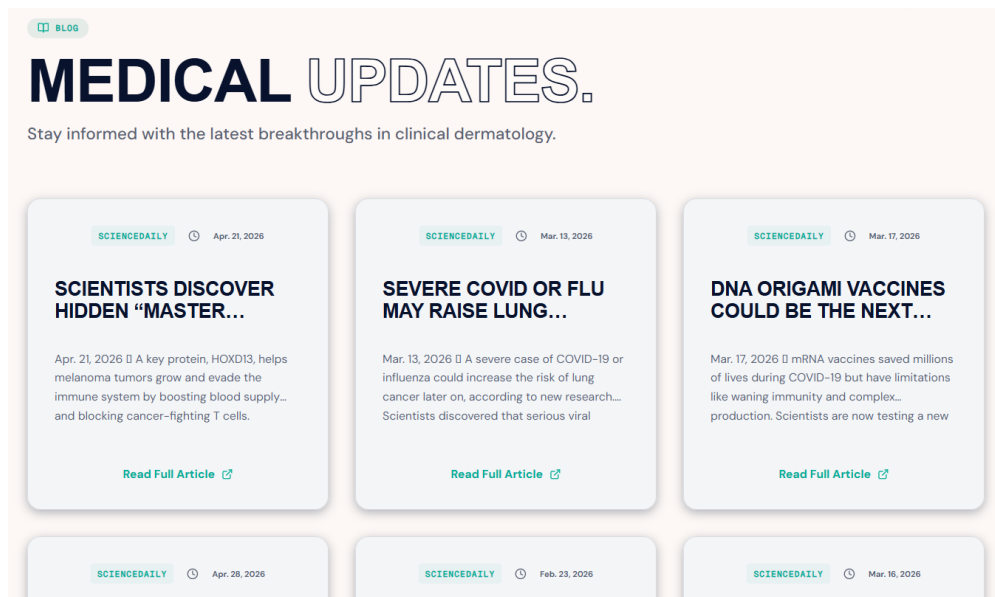


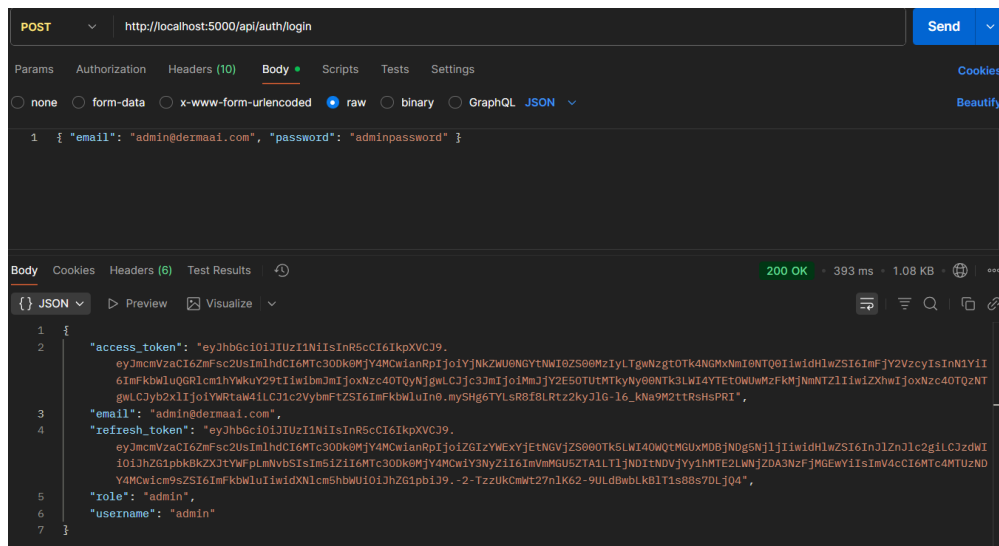
Figure 3.8: Dynamic Medical News Interface

3.6.2. API Testing

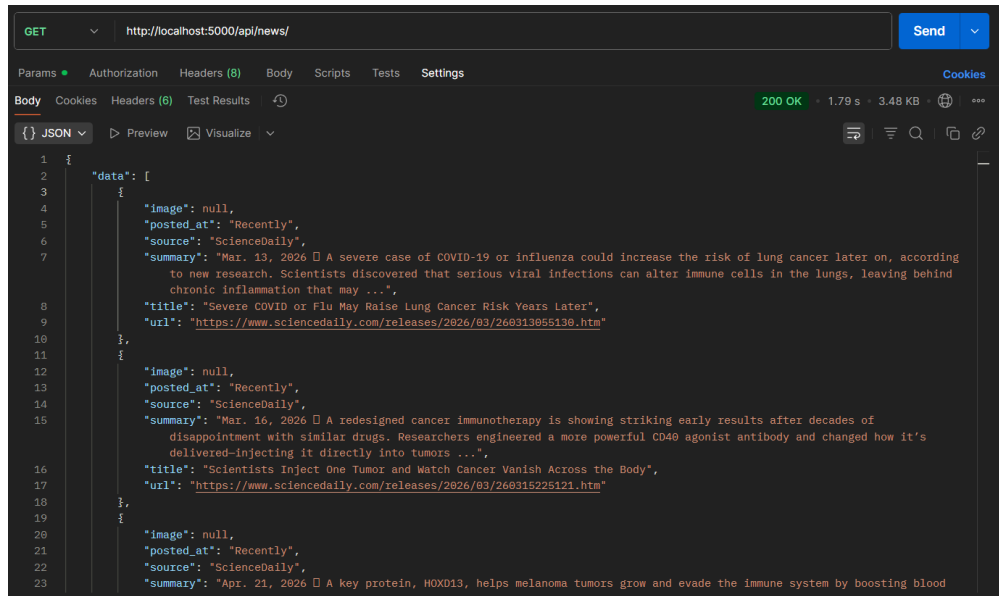
API testing was conducted using Postman to validate the robustness of the backend services.

Authentication Endpoint Validation

The `/api/auth/login` route was tested under nominal and alternative conditions. Supplying valid credentials successfully returns a JSON response containing the generated JWT access token and user role (Status 200 OK). Supplying incorrect credentials returns a secure “Invalid credentials” message without revealing whether the email exists (Status 401 Unauthorized). Figure 3.9 illustrates the Postman test results.



News Web Scraper Validation



3.7. Progress Tracking

To ensure agile delivery, task progress was rigorously tracked using Trello Kanban boards. Figure 3.11 displays the state of the sprint board upon the completion of Sprint 1 tasks, visually confirming the realization of the planned backlog.

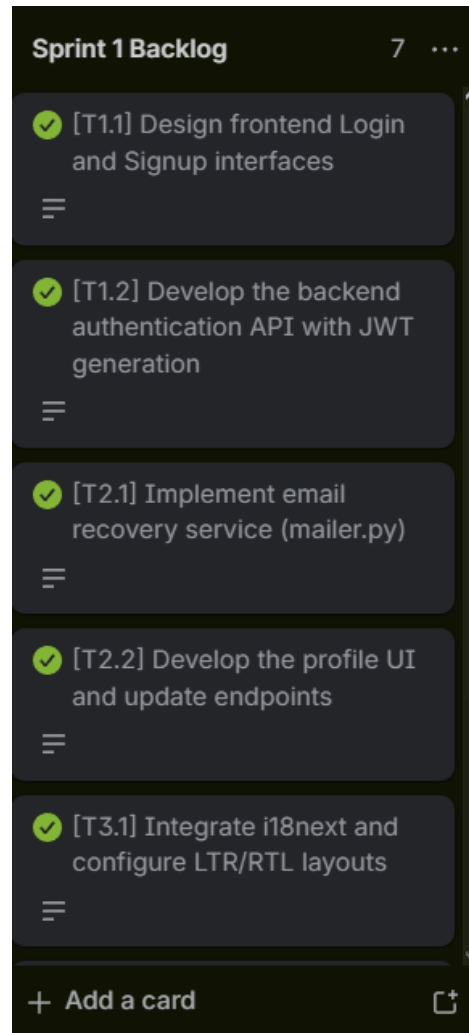


Figure 3.11: Trello Board at the End of Sprint 1

3.8. Conclusion

This chapter successfully concluded Sprint 1 by establishing the core security and content delivery architecture of DermaAI. Secure authentication, multilingual support, and a dynamic news scraper were fully developed and validated. With the identity layer in place, the platform is now ready for the integration of its core AI-powered diagnostic engine in the next phase.

Chapter 4: Sprint 2 — AI Scan & Diagnosis



4.1. Introduction

This chapter presents the second development sprint, focusing on the core AI-powered skin lesion scanner. The diagnostic pipeline leverages the Xception convolutional neural network, trained on the HAM10000 dataset and enhanced with Grad-CAM explainability. The integration tasks, frontend scanning interface, and detailed AI model evaluations are documented in the following sections.

4.2. Objectives

The main objectives to be accomplished during this sprint are:

- Develop an intuitive frontend interface allowing users to upload skin lesion images.
- Implement an interactive body map for users to optionally pinpoint the location of the lesion, providing contextual data for the scan.
- Integrate the trained Xception deep learning model to return a primary and secondary diagnosis with confidence scores.
- Implement Grad-CAM (Gradient-weighted Class Activation Mapping) to generate visual heatmaps, explaining exactly which regions of the image the AI analyzed.
- Automatically store the scan results and contextual data in the user's history.

4.3. Design

This section details the functional and static design of the core scanning process.

4.3.1. Use-Cases Modeling

Figure 4.1 illustrates the use-cases diagram for the scanning process.

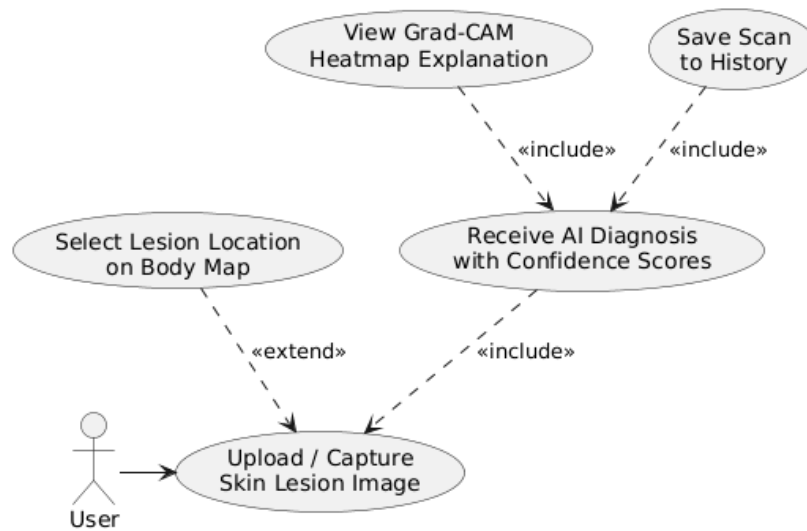


Figure 4.1: Use-Cases Diagram for Sprint 2

4.3.1.1. Perform Skin Lesion Scan

Use Case	Perform Skin Lesion Scan
Actor	User
Precondition	The user is authenticated and on the Scan page.
Postcondition	The AI returns a diagnosis, a Grad-CAM heatmap, and saves the record.
Nominal Scenario	1. The user uploads an image of a skin lesion. 2. (Optional) The user selects the affected area on the body map. 3. The system processes the image using the Deep Learning model. 4. The system generates a Grad-CAM heatmap. 5. The results (diagnosis, confidence, heatmap) are displayed. 6. The record is stored in the database history.
Alternative	If the uploaded file is corrupted or not a valid image format, the system rejects it and displays an error alert.

Table 4.1: Textual Description: Perform Skin Lesion Scan

4.3.2. Activity Diagram

Figure 4.2 presents the activity diagram modeling the scanning process.

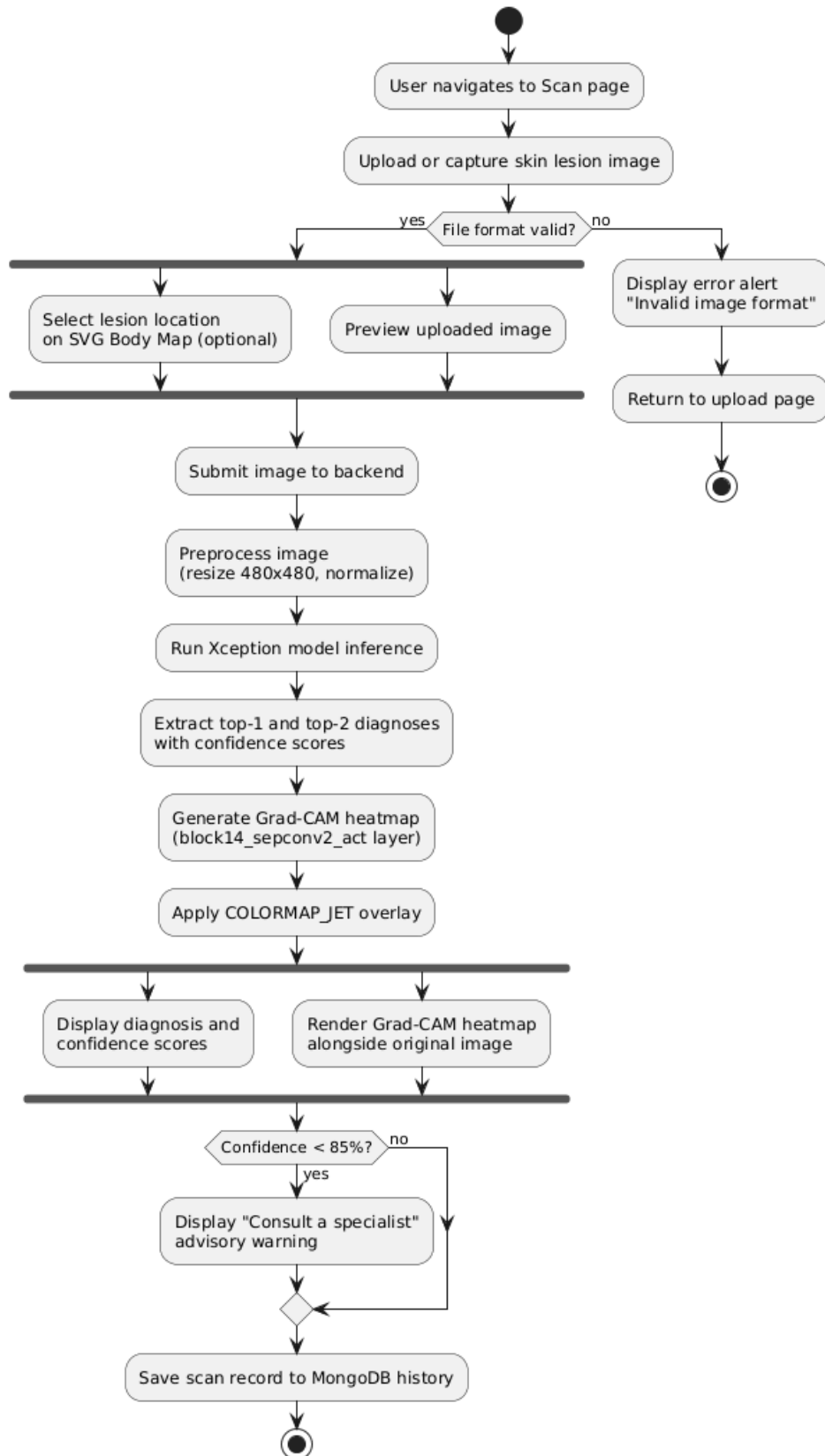


Figure 4.2: Activity Diagram for Scanning Process

4.3.3. Class Diagram

Figure 4.3 shows the conceptual class diagram for the entities managed during this sprint.

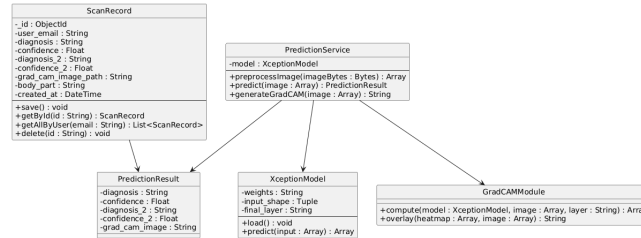


Figure 4.3: Conceptual Class Diagram for Sprint 2

4.3.4. Sequence Diagram

Figure 4.4 outlines the interactions between the React client, the Flask API, the Deep Learning Prediction Service, and the MongoDB database during an active scan request.

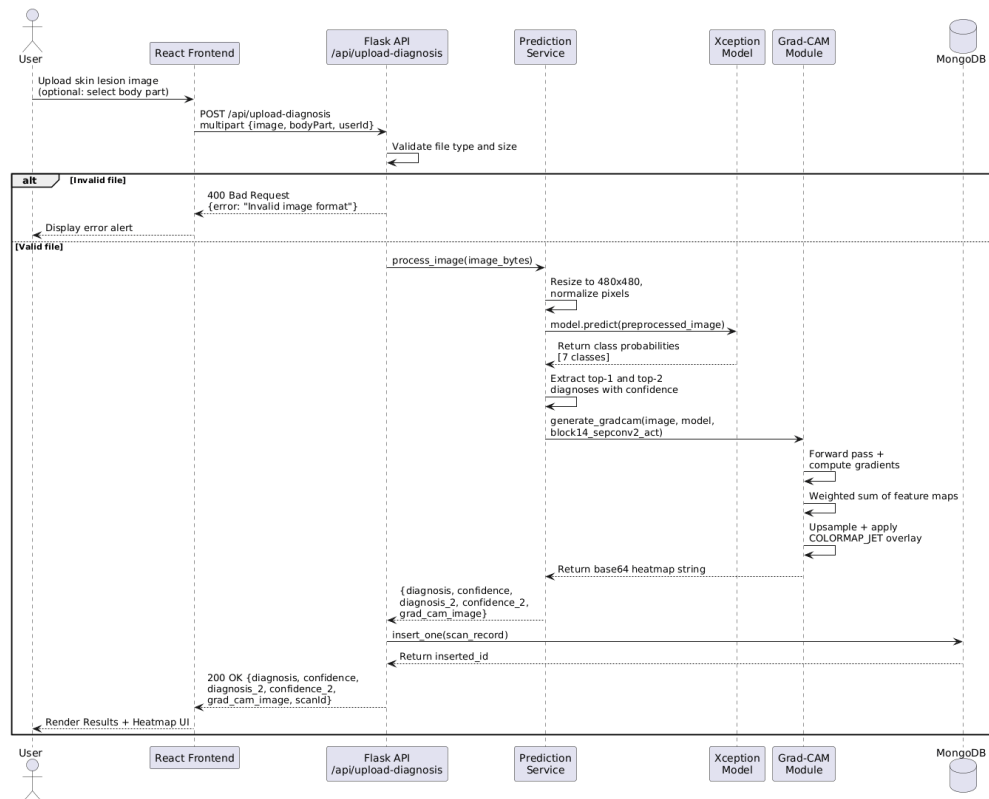


Figure 4.4: Sequence Diagram: Scanning and Prediction Flow

4.4. Tasks

The specific development tasks required to fulfill the objectives of Sprint 2 are defined in the sprint backlog below.

ID	User Story	Task Description
5	As a user, I want to upload/capture a skin lesion image and optionally pinpoint its location on a body map.	Design the Image Upload component (drag/drop and camera capture).
		Develop the interactive SVG Body Map selector.
6	As a user, I want to receive an AI-powered preliminary diagnosis with confidence scores.	Develop the Flask prediction route (/upload-diagnosis).
		Integrate the pre-trained Xception model into the Flask backend.
7	As a user, I want to view a visual heatmap over my image to understand exactly which areas the AI analyzed.	Implement the Grad-CAM extraction script targeting the final convolutional layer.
		Develop the Results Display UI to render the diagnoses and base64 heatmap overlays.

Table 4.2: Sprint 2 Backlog

4.5. Development

This section showcases the developed user interfaces for the scanning process and validates the backend logic through API testing.

4.5.1. User Interfaces

Scan and Body Map Interface

The upload interface offers a clean drag-and-drop zone. Alongside the image upload, users interact with an SVG-based Body Map component to optionally select the general location of the lesion, which provides helpful medical context. Figure 4.5 and Figure 4.6 present the interactive body map and the scan interface, respectively.

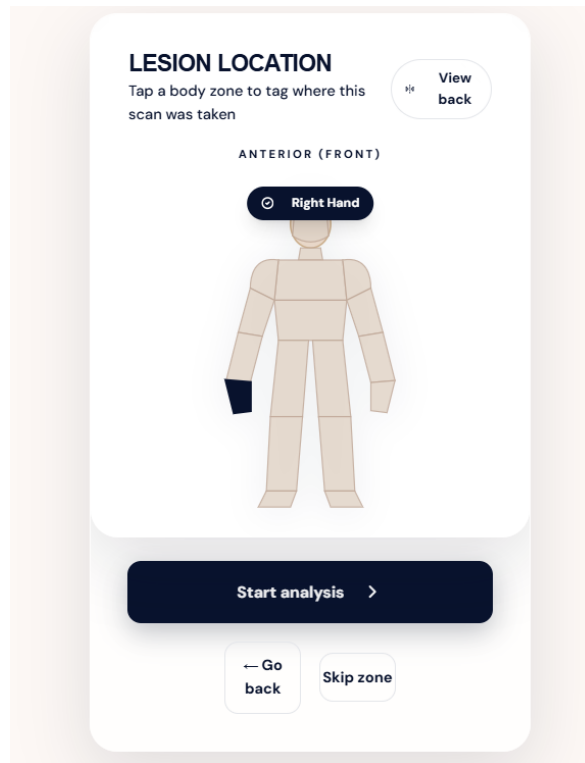


Figure 4.5: Interactive Body Map for Scan Location

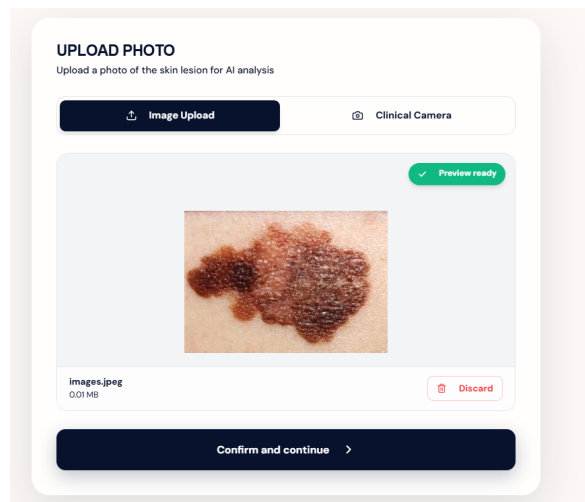


Figure 4.6: Skin Lesion Image Upload Interface

Results and Explanation Interface

Once the prediction is processed, the system transitions to the Results Display. This interface prominently shows the uploaded image alongside the generated Grad-CAM heatmap, making the AI's focal points visually explicit. It also details the primary and secondary diagnoses with their respective confidence scores. Figure 4.7 shows this results interface.

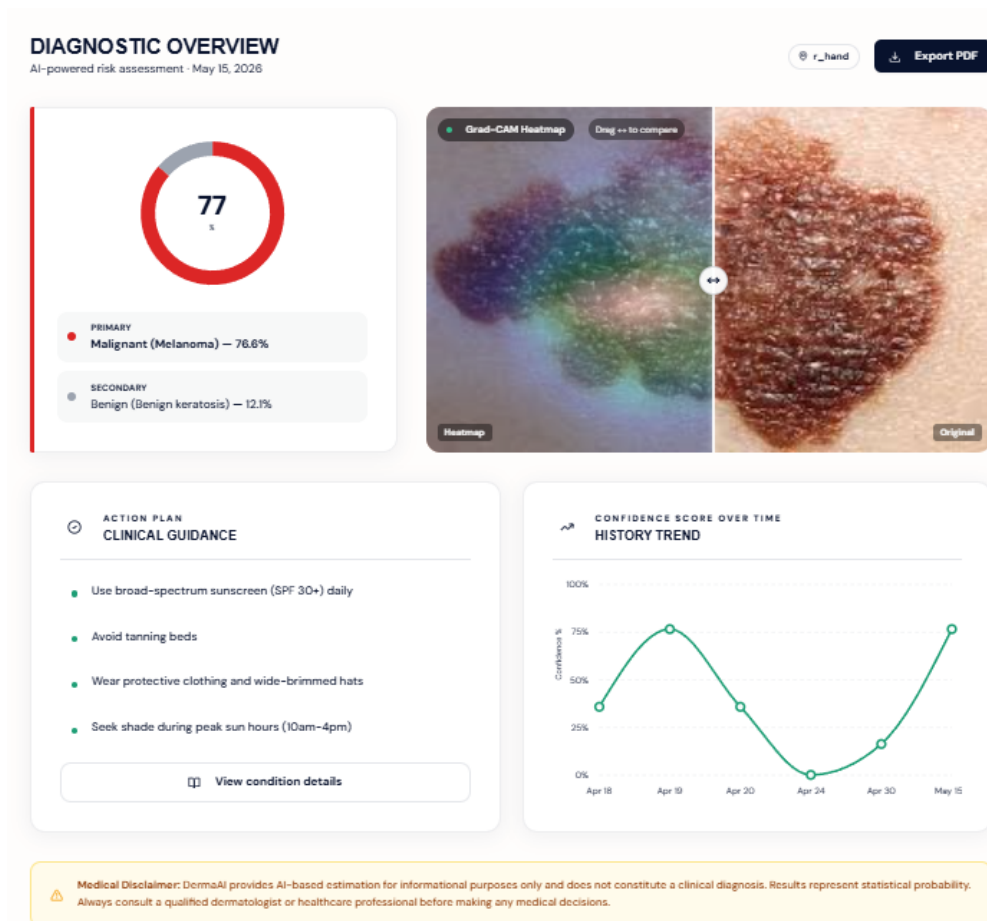


Figure 4.7: Diagnostic Results Display

4.5.2. API Testing

The deep learning pipeline was verified using Postman by targeting the `/api/upload-diagnosis` endpoint.

Inference Endpoint Validation

Uploading a valid dermoscopic image via a `multipart/form-data` POST request successfully triggers the prediction service. The endpoint processes the image through the Xception model and returns a JSON payload containing the `diagnosis`, `confidence`, `diagnosis_2`, and a `grad_cam_image` formatted as a Base64 string (Status 200 OK). Figure 4.8 shows the Postman response from the inference endpoint.

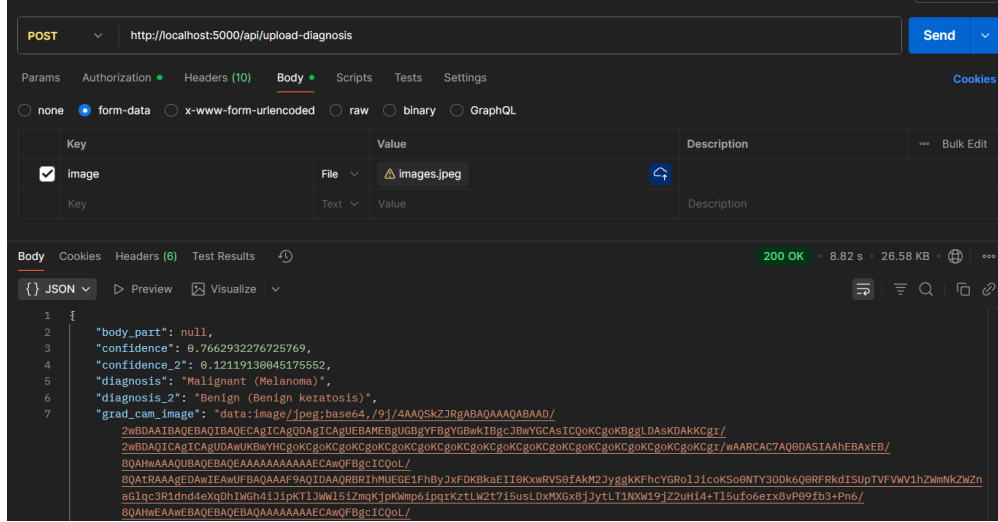


Figure 4.8: Postman Test: Deep Learning Inference API

4.6. AI Model Development

The intelligence of the DermaAI platform relies on a highly optimized Convolutional Neural Network pipeline. This section provides an in-depth account of all decisions made during the design, training, and evaluation phases, grounded in the actual experimental results obtained from the training notebook [14, 21].

4.6.1. Dataset: HAM10000

The model was trained on the **HAM10000** (Human Against Machine with 10,000 training images) dataset [14], the internationally recognized benchmark for dermatological AI research. It contains **10,015** dermoscopic images across **7 clinically relevant diagnostic classes**. The dataset was partitioned using stratified sampling to preserve class distribution ratios across all splits, as summarized in Table 4.3.

Split	Images	Ratio	Purpose
Training	8,012	80%	Model weight optimization
Validation	1,001	10%	Hyperparameter tuning & early stopping
Test	1,002	10%	Final unbiased evaluation
Total	10,015	100%	

Table 4.3: HAM10000 Dataset Split Distribution

The class distribution within the raw training set is severely imbalanced, as shown in Table 4.4. The dominant class (NV — Melanocytic Nevi) accounts for 66.95% of

training samples, while rare classes such as Dermatofibroma (DF) and Vascular Lesions (VASC) represent less than 2% each. A naive model trained on this distribution would be strongly biased toward the majority class, producing clinically unreliable results for rare but dangerous conditions.

Code	Class Name	Train Samples	% of Training Set
NV	Melanocytic Nevi (Benign mole)	5,364	66.95%
MEL	Melanoma (Malignant)	890	11.11%
BKL	Benign Keratosis-like Lesions	879	10.97%
BCC	Basal Cell Carcinoma	411	5.13%
AKIEC	Actinic Keratoses	262	3.27%
VASC	Vascular Lesions	114	1.42%
DF	Dermatofibroma	92	1.15%

Table 4.4: Raw Class Distribution in the Training Set

Figure 4.9 presents representative dermoscopic images from each of the seven diagnostic classes, illustrating the high degree of visual similarity between several malignant and benign lesion types that makes this classification task clinically challenging.

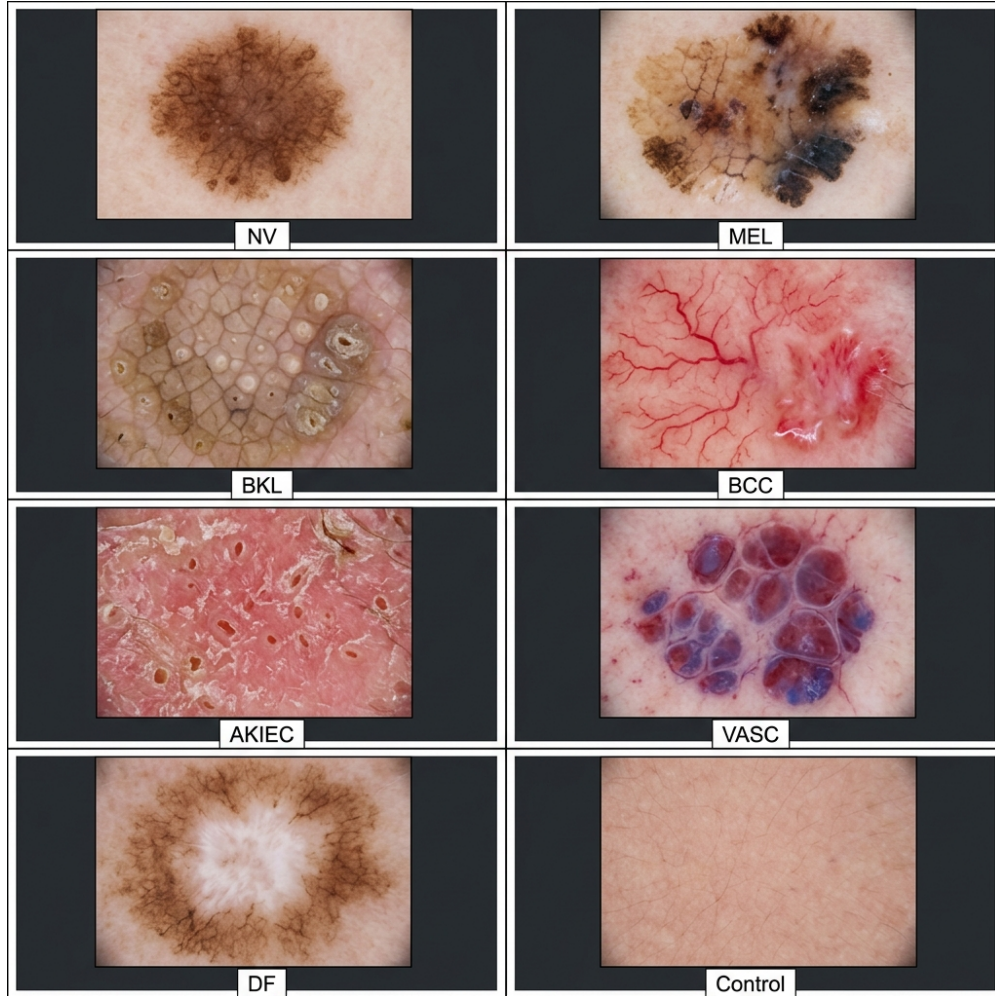


Figure 4.9: Representative Samples for the Seven Diagnostic Classes

The sample distribution across each of the seven diagnostic classes illustrates the severe class imbalance and the high dominance of the NV class.

4.6.2. Class Imbalance Strategy

To counteract the severe class imbalance, a data-level oversampling strategy was implemented via a custom `balance()` function. A target of **1,006 samples per class** was established. For all minority classes, the function uses Keras `ImageDataGenerator` to synthesize augmented copies using the following transformations:

- Horizontal flips.
- Random rotations up to 20° .
- Width and height shifts up to 20%.
- Zoom augmentation up to 20%.

After balancing, the effective training set grew from 8,012 raw images to **7,042**

balanced images (7 classes $\times$ 1,006 samples each). All images were resized to a fixed resolution of **480 $\times$ 480 pixels** with 3 RGB channels, providing sufficient spatial detail for the depthwise separable convolutions to extract fine-grained skin texture features.

4.6.3. Architecture Selection: Xception Model

The architecture selection was made following a comparative evaluation of leading CNN models available in the Keras Applications module (such as VGG16, ResNet50, and EfficientNet). **Xception** [15] was ultimately selected for the following reasons:

- **Depthwise Separable Convolutions:** Xception decomposes standard convolutions into separate depthwise and pointwise steps, reducing parameter count while improving representational capacity — critical for distinguishing subtle texture differences between malignant and benign lesions.
- **ImageNet Pre-trained Weights:** The backbone was initialized with ImageNet weights, providing rich generic feature representations (edges, gradients, textures) that transfer effectively to the medical imaging domain.
- **Grad-CAM Compatibility:** The architecture’s final convolutional layer (`block14_sepconv2_act`) produces spatially meaningful 2D feature maps that directly feed the Grad-CAM localization algorithm.

A custom classifier head was appended to the Xception backbone: a **BatchNormalization** layer, followed by two **Dense** layers (256 and 128 neurons) with **Swish** activation, L2 regularization ($\lambda = 0.0016$), and **Dropout** (rates 0.5 and 0.2 respectively), terminated by a 7-neuron **Softmax** output layer.

4.6.4. Training Configuration & Adaptive LR Strategy

The model was compiled using the **AdamW** optimizer (Adam with decoupled weight decay), which prevents weight decay from interfering with the gradient update step and generally improves generalization [21]. The model was trained using Categorical Cross-Entropy loss, a batch size of 16, and an initial learning rate of 0.001 over a maximum of 50 epochs.

A custom **Learning Rate Annealing (LRA)** callback was implemented to dynamically adapt the learning rate. When training accuracy is below 90%, the callback monitors training accuracy. Once the 90% threshold is surpassed, the monitor switches to **validation loss**, preventing the model from overfitting to the training distribution

during the fine-tuning phase.

4.6.5. Training Progress

Training ran for **18 epochs** before the early stopping criterion was triggered (3 consecutive learning rate adjustments without validation improvement). The total training duration was **1 hour, 22 minutes, and 3.89 seconds**, with each epoch averaging approximately 271 seconds.

Figure 4.10 shows the complete training and validation loss and accuracy progression across all 18 epochs, clearly showing convergence and the effect of adaptive learning rate reductions.

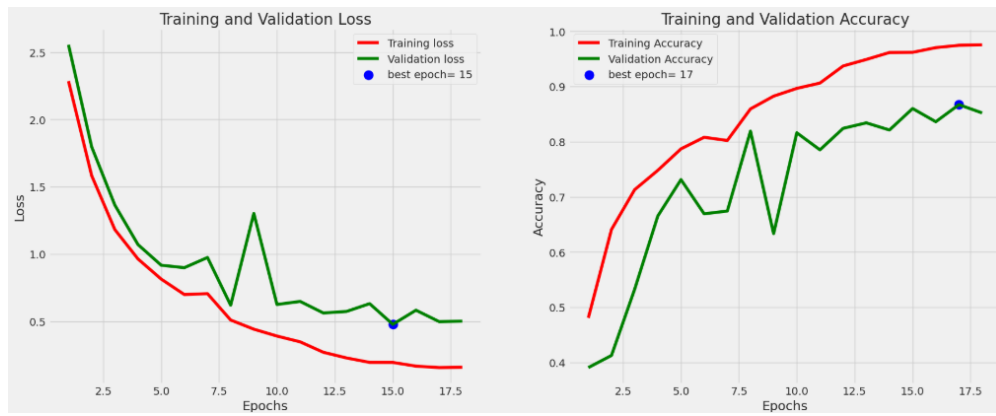


Figure 4.10: Training and Validation Loss & Accuracy Curves (18 Epochs)

4.6.6. Evaluation on the Test Set

The final model was evaluated on the held-out test set of **1,002 images**, achieving an overall test accuracy of **83.83%**. The detailed per-class performance metrics are presented in Table 4.5.

Class	Precision	Recall	F1-Score	Test Samples
AKIEC	0.71	0.69	0.70	32
BCC	0.83	0.85	0.84	52
BKL	0.64	0.91	0.75	110
DF	0.67	0.36	0.47	11
MEL	0.54	0.62	0.58	112
NV	0.96	0.87	0.91	671
VASC	0.93	1.00	0.97	14
Weighted Avg	0.86	0.84	0.84	1,002
Macro Avg	0.75	0.76	0.75	1,002

Table 4.5: Per-Class Classification Report on the 1,002-Image Test Set

Key observations from the results include:

- **NV** achieves the highest F1-score (0.91) due to its dominant representation in the dataset.
- **VASC** achieves perfect recall (1.00), demonstrating that the augmentation strategy successfully compensated for its very limited original sample count (114 training images).
- **MEL (Melanoma)** achieves a recall of 0.62. From a clinical safety perspective, this is the most critical metric: a missed melanoma is far more dangerous than a false positive. The platform therefore displays an explicit “Consult a specialist” advisory for all predictions with confidence below 85%.
- **DF (Dermatofibroma)** has the lowest performance ($F1 = 0.47$) due to its very limited support (only 11 test samples), making statistical evaluation unreliable for this class.

Figure 4.11 presents the confusion matrix, providing a visual breakdown of correct predictions versus misclassifications across all seven classes.

Confusion Matrix								
Actual	AKIEC	22	2	7	0	1	0	0
	BCC	2	44	3	0	1	2	0
	BKL	1	2	100	0	5	2	0
	DF	2	0	2	4	0	3	0
	MEL	1	0	22	0	69	20	0
	NV	3	5	22	2	51	587	1
	VASC	0	0	0	0	0	0	14
		AKIEC	BCC	BKL	DF	MEL	NV	VASC
		Predicted						

Figure 4.11: Confusion Matrix on the 1,002-Image Test Set

4.6.7. Model Explainability: Grad-CAM

To establish clinical trust, the platform does not operate as a “black box.” The **Gradient-weighted Class Activation Mapping (Grad-CAM)** algorithm [13] was implemented. The process involves: (1) a forward pass through the full Xception network; (2) extraction of the spatial gradient of the predicted class score with respect to the output of the final convolutional layer (`block14_sepconv2_act`); (3) computation of a weighted sum of the feature maps using these gradients as importance weights; (4) bilinear upsampling to the original image resolution; and (5) application of a `COLORMAP_JET` color overlay where red areas indicate the regions most influential for the prediction.

4.6.8. Inference Performance

The deployed model is loaded once at Flask server startup (taking approximately 10 seconds) and held in memory. During active operation, the total API response time (end-to-end), including image preprocessing, Xception inference, and Grad-CAM gen-

eration, averages under 2 seconds.

4.7. Progress Tracking

Task execution was monitored via the Trello Kanban board. Figure 4.12 captures the state of the board at the completion of the sprint, verifying that all AI integration and frontend scanning tasks were moved to the "Done" list.

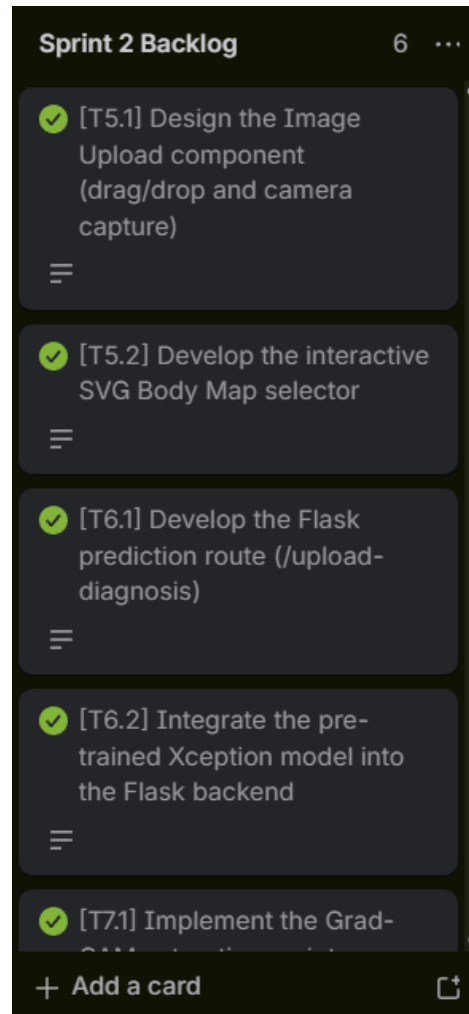


Figure 4.12: Trello Board at the End of Sprint 2

4.8. Conclusion

Sprint 2 successfully delivered the core diagnostic functionality of the DermaAI platform through a fine-tuned Xception model. By pairing high-accuracy predictions with transparent Grad-CAM visualizations, the system provides explainable and trustworthy assessments. With the scanning engine operational, the next chapter focuses on expanding user assistance through an AI chatbot.

Chapter 5: Sprint 3 — History, AiChabot & Reports



5.1. Introduction

This chapter details the third development sprint, focusing on enriching the post-diagnostic user experience. It covers the integration of a localized Qwen conversational assistant designed to answer dermatological questions in complete privacy. Additionally, the implementation of a scan history dashboard and a structured PDF report generator is presented and explained.

5.2. Objectives

The primary objectives established for Sprint 3 include:

- Implement a secure, localized AI Chatbot capable of answering user queries while maintaining strict medical safety boundaries.
- Develop an interface allowing users to view, manage, and delete their past scan histories from the database.
- Create a dynamic PDF generator to compile the AI diagnosis, confidence scores, and Grad-CAM visualizations into a structured, downloadable medical report.
- Enable longitudinal tracking to allow users to compare their baseline scans with follow-up scans over time.

5.3. Design

This section outlines the functional and interactive design of the assistance and tracking tools developed during the sprint.

5.3.1. Use-Cases Modeling

Figure 5.1 illustrates the use-cases for Sprint 3, demonstrating how the User interacts with the Chatbot and the History management system.

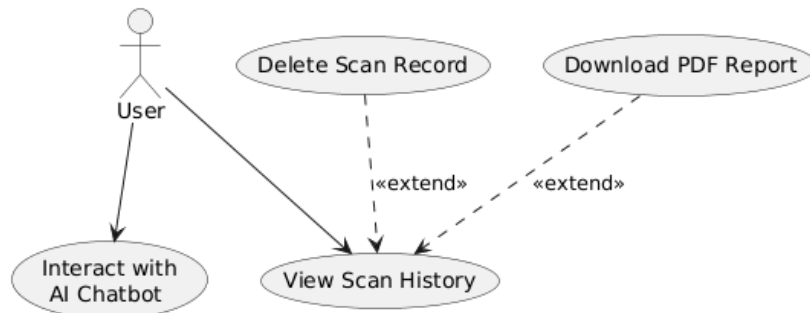


Figure 5.1: Use-Cases Diagram for Sprint 3

5.3.1.1. Interact with Digital Assistant

Use Case	Interact with Digital Assistant
Actor	User
Precondition	The user is logged in and opens the Chatbot interface.
Postcondition	The assistant provides a context-aware response based on the latest scan.
Nominal Scenario	1. The user asks a question regarding their skin condition. 2. The frontend sends the query along with the recent diagnosis context to the backend. 3. The local LLM (Large Language Model) processes the prompt. 4. The assistant streams the generated text back to the user interface.
Alternative	If the user asks an off-topic question (e.g., politics, coding), the assistant politely declines and redirects the conversation back to dermatology.

Table 5.1: Textual Description: Interact with Digital Assistant

5.3.2. Activity Diagram

Figure 5.2 presents the activity diagram modeling the user interactions during Sprint 3.

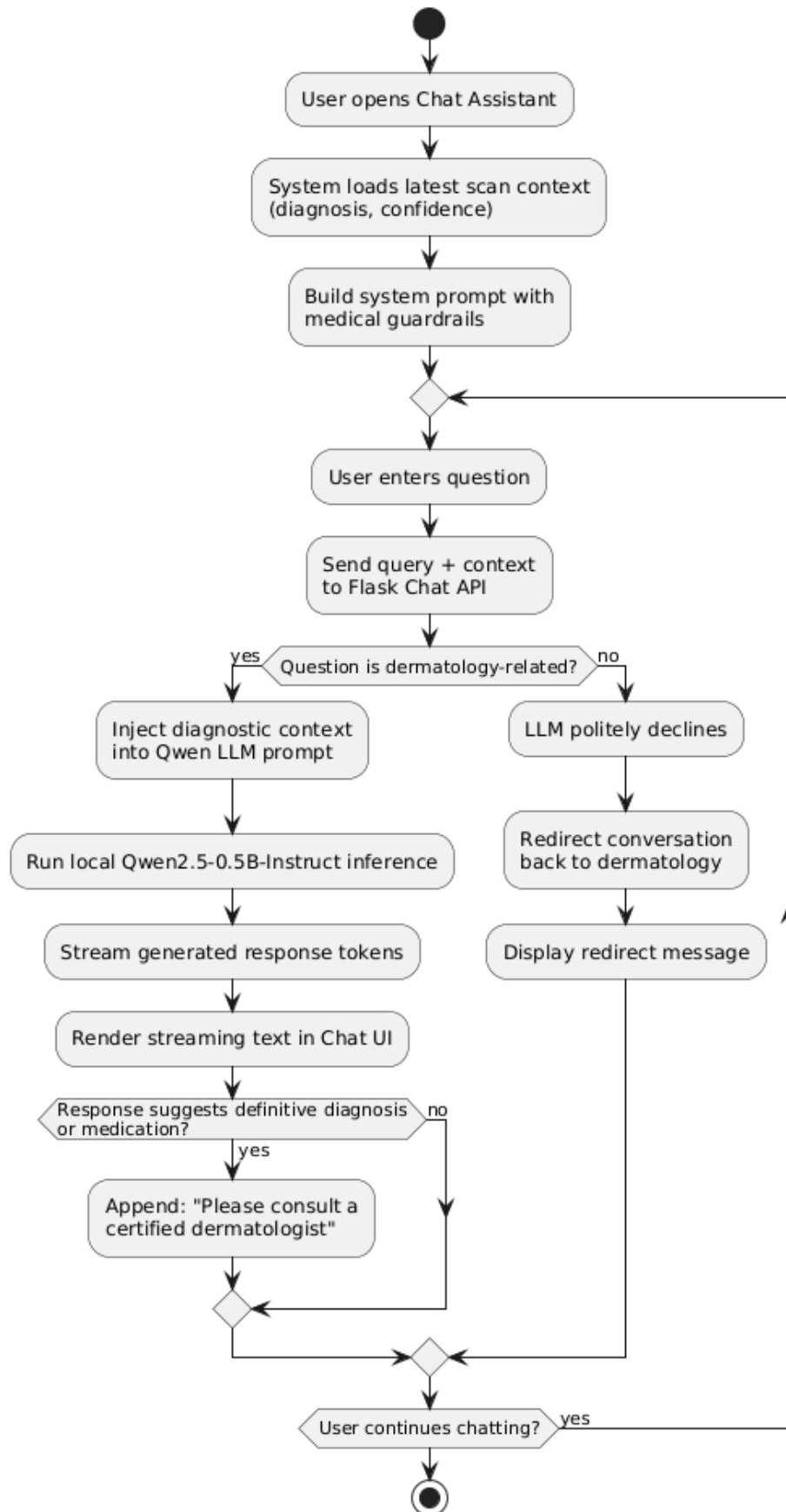


Figure 5.2: Activity Diagram for Sprint 3

5.3.3. Class Diagram

Figure 5.3 shows the conceptual class diagram for the entities managed during this sprint.

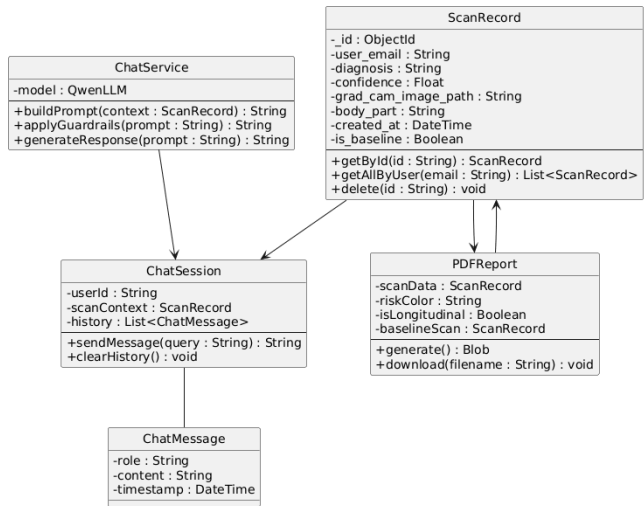


Figure 5.3: Conceptual Class Diagram for Sprint 3

5.3.4. Sequence Diagram

Figure 5.4 maps the interaction flow between the client, the backend Chat API, and the local HuggingFace inference engine when generating a conversational response.

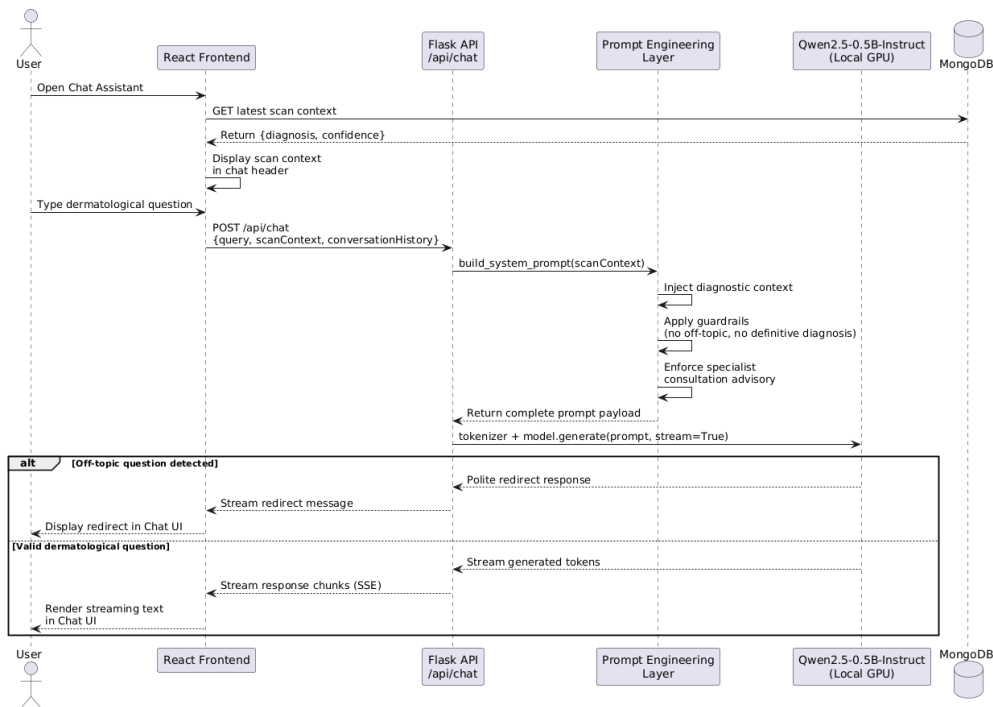


Figure 5.4: Sequence Diagram: AI Chatbot Interaction

5.4. Tasks

The specific implementation tasks required to complete Sprint 3 are detailed in the sprint backlog.

ID	User Story	Task Description
8	As a user, I want to interact with a smart digital assistant to ask follow-up questions about my screening results.	Configure the backend to load the local Qwen language model using PyTorch/Transformers.
		Develop the Floating Chat UI component with real-time text streaming.
9	As a user, I want to view my scan history and delete my personal records if needed.	Create backend REST endpoints to retrieve and delete documents from the MongoDB history collection.
		Design the History Dashboard UI with a responsive grid.
10	As a user, I want to generate and download a comprehensive PDF report of my scan for my medical records.	Implement client-side PDF generation using jsPDF, injecting dynamic risk colors and Grad-CAM overlays.
		Implement the Longitudinal Comparison PDF logic for baseline follow-ups.

Table 5.2: Sprint 3 Backlog

5.5. Development

This section highlights the user interfaces and functional systems developed for user assistance and tracking.

5.5.1. User Interfaces

Floating Chat Assistant Interface

The digital assistant is integrated as a floating module, ensuring it is universally accessible across the platform. The UI supports real-time text streaming, giving the conversational AI a responsive, human-like typing feel. Figure 5.5 shows the chat assistant interface.

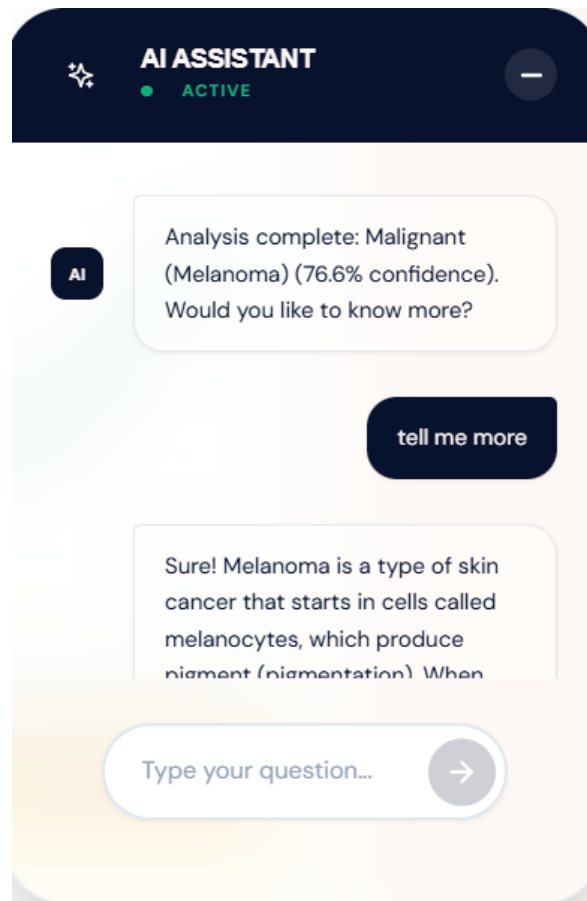


Figure 5.5: Digital Assistant Chat Interface

Scan History Interface

The History page retrieves the user's past scans from MongoDB and displays them chronologically in a clear grid format. Users can directly download PDF reports for past scans, delete specific records to maintain privacy, or select two scans for longitudinal comparison. Figure 5.6 presents the scan history interface.

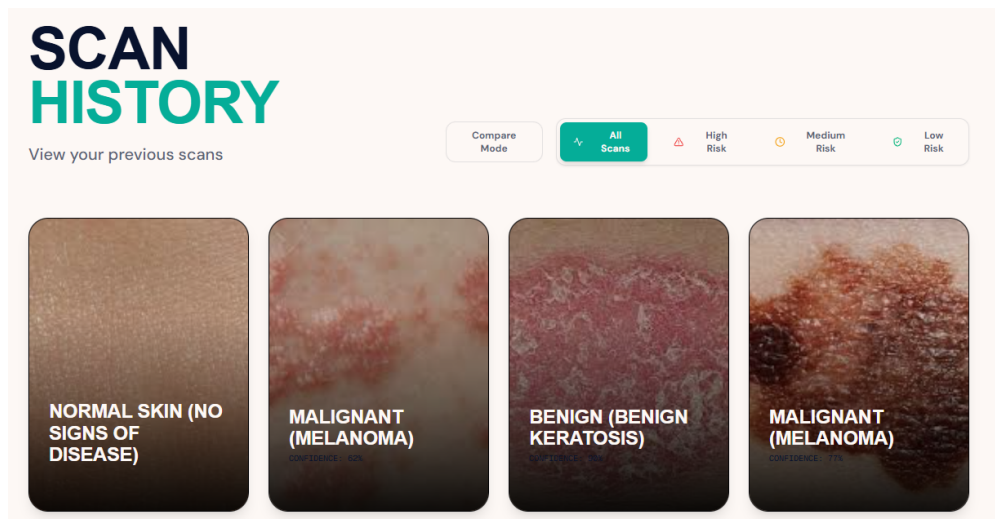


Figure 5.6: Scan History Management Interface

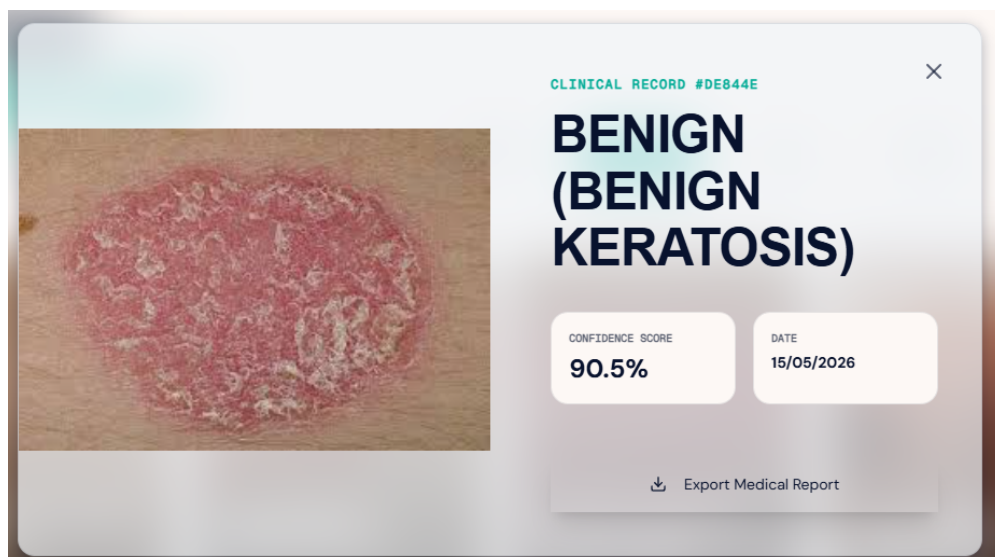


Figure 5.7: Individual Scan History Details

Generated PDF Medical Report

The system utilizes jsPDF to generate structured, professional documents on the client side. The PDFs are dynamically styled, featuring risk-based color coding (e.g., Amber for moderate risk, Red for high risk), the embedded original image, and clear clinical guidance with a strict medical disclaimer. Figure 5.8 shows an example of a generated diagnostic report.

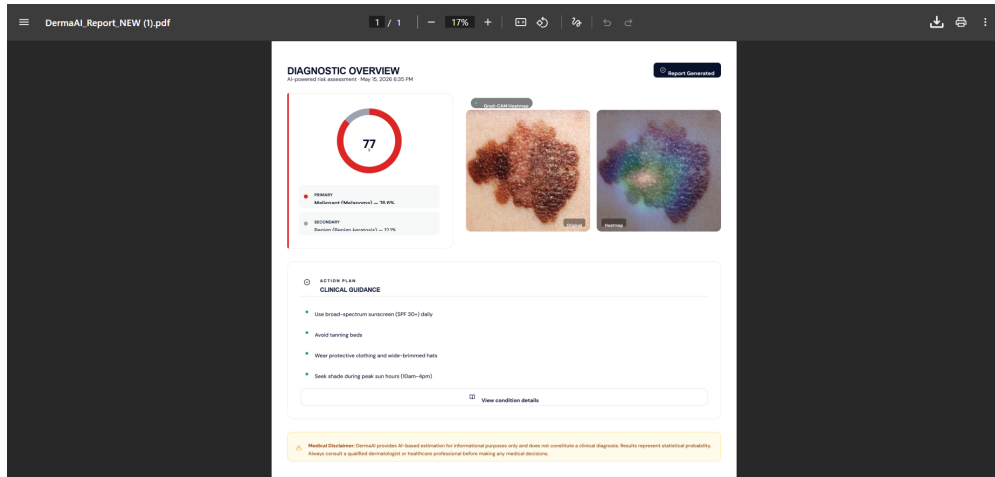


Figure 5.8: Example of a Generated Diagnostic PDF Report

5.6. AI Model Development: Local Conversational Assistant

One of the most critical architectural decisions for the Chatbot was prioritizing patient data privacy. Instead of relying on external commercial APIs (which would require sending sensitive medical inquiries to third-party servers), a **100% Local Large Language Model (LLM)** was integrated.

5.6.1. Model Selection and Inference Engine

The system implements the **Qwen2.5-0.5B-Instruct** model, loaded dynamically via the HuggingFace `transformers` and `torch` libraries. This lightweight but highly capable instruct-tuned model is executed directly on the host server's GPU, guaranteeing that user queries and diagnostic contexts remain entirely private.

To optimize system memory, the chatbot service was implemented using a **Singleton Design Pattern**. This ensures that the 0.5B parameter model and its tokenizer are only loaded into GPU memory once during the application's lifecycle. A fallback mechanism was also implemented: if the heavy AI libraries are unavailable or fail to load, the system degrades gracefully and returns a predefined localized error message, allowing the rest of the web application to function without crashing.

5.6.2. Contextual Prompt Engineering and Localization

To ensure the AI operates safely within a medical context, rigorous system-level Prompt Engineering was applied. The model is injected with a hidden context payload during

every interaction containing:

- **Diagnostic Context:** If the user just completed a scan, the exact Xception prediction (e.g., "Actinic Keratosis, 85% confidence") is fed to the LLM, allowing it to provide specific, relevant advice regarding their exact result.
- **Topic Guardrails:** The system prompt explicitly strictly forbids the model from discussing non-medical topics (e.g., politics, coding) and forces it to politely redirect the conversation back to dermatology.
- **Medical Safety Constraints:** The model is explicitly barred from generating definitive diagnoses or suggesting medication, actively enforcing a rule to encourage users to consult certified dermatologists.
- **Dynamic Localization:** The prompt dynamically injects strict language enforcement rules (e.g., "*You MUST respond ONLY in Arabic*"). This allows the model to reply natively in English, French, or Arabic based on the user's active UI preference, entirely bypassing the need for external translation services.

5.6.3. Real-Time Token Streaming

Generating responses from an LLM can take several seconds. To prevent the user interface from freezing while waiting for the full response, a real-time streaming architecture was implemented. Using the `TextIteratorStreamer` utility combined with Python's `threading` module, the backend begins generating tokens asynchronously. As each word is predicted by the model, it is immediately yielded via a Server-Sent Events (SSE) stream to the React frontend. This provides a fast, human-like typing experience, significantly improving the perceived performance of the assistant.

5.7. Progress Tracking

Task execution was monitored via the Trello Kanban board. Figure 5.9 captures the state of the board at the completion of Sprint 3, verifying that all assistant and tracking functionalities were realized.

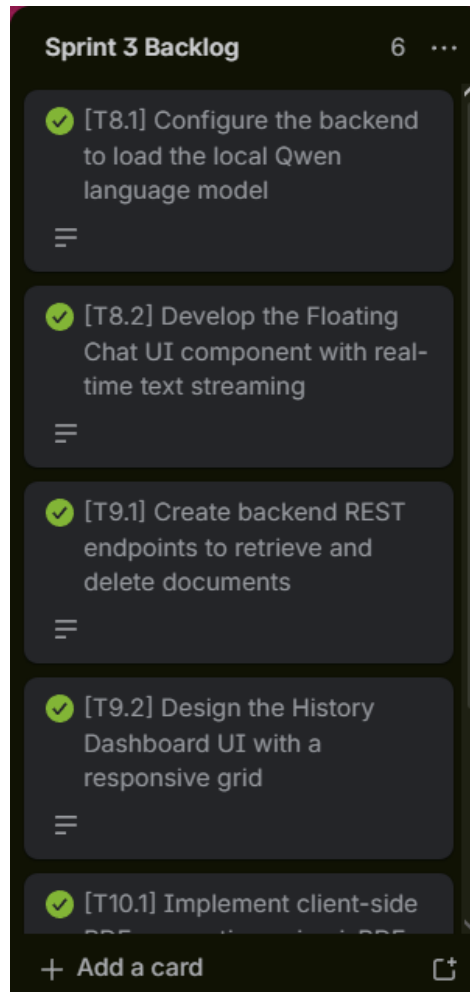
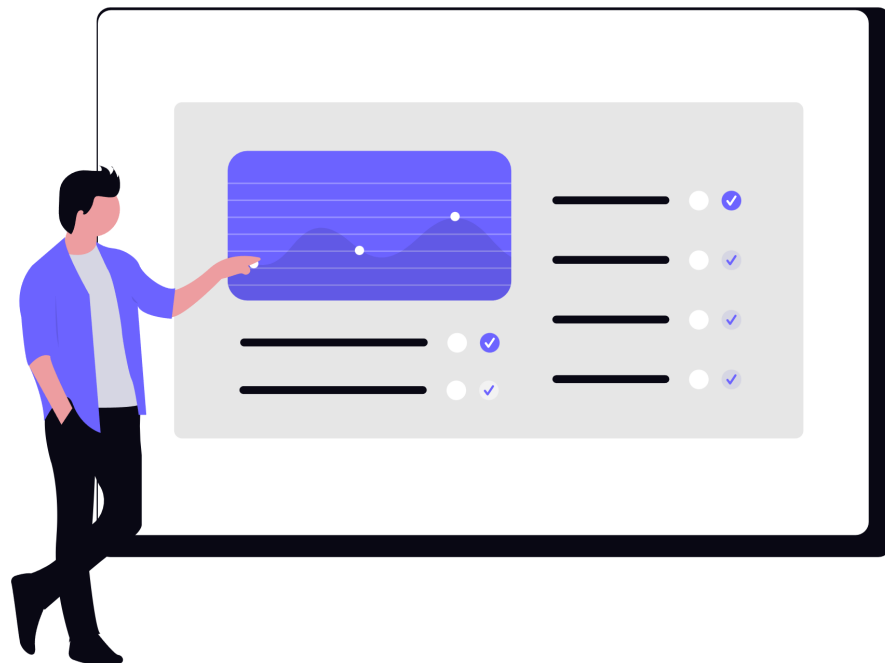


Figure 5.9: Trello Board at the End of Sprint 3

5.8. Conclusion

Sprint 3 successfully bridged the gap between AI predictions and actionable user insights through personalized assistance tools. By implementing a privacy-preserving chatbot, chronological scan histories, and professional PDF exports, the system provides a reliable way to track dermatological health. The final chapter will explore Sprint 4, which focuses on administrative controls and analytics.

Chapter 6: Sprint 4 — Admin Dashboard & Data Export



6.1. Introduction

This chapter presents the final development sprint, focusing on the platform’s administrative infrastructure and data security. It covers the development of a real-time metrics dashboard, a secure user management interface, and a robust data anonymization pipeline for research exports. The design, Development, and security architecture of these features are documented below.

6.2. Objectives

The main objectives established for Sprint 4 are:

- Develop an Admin Dashboard to visualize real-time platform statistics (active users, total scans, diagnosis distribution).
- Implement a secure user management interface allowing administrators to view, update, and remove accounts.
- Create a full database backup endpoint to export JSON configurations of the entire system.
- Develop an anonymized dataset export tool to safely extract scanning metrics for future AI model retraining without exposing Personally Identifiable Information (PII).

6.3. Design

This section outlines the functional design of the administrative operations.

6.3.1. Use-Cases Modeling

Figure 6.1 illustrates the specific administrative use cases, which are strictly isolated from standard user operations.

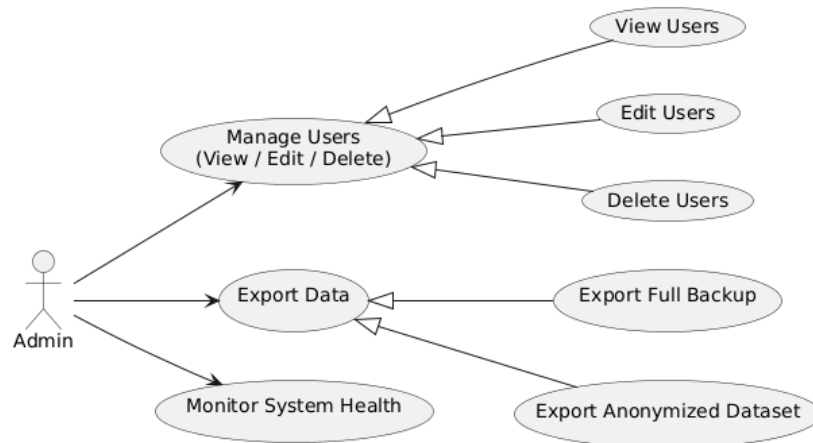


Figure 6.1: Use-Cases Diagram for Sprint 4 (Admin Actor)

6.3.1.1. Export System Data

Use Case	Export System Data
Actor	Administrator
Precondition	The admin is securely authenticated and possesses the "admin" JWT role claim.
Postcondition	The system securely generates and downloads a JSON export file.
Nominal Scenario	1. The administrator navigates to the Exports tab. 2. The admin selects either "Full Backup" or "Anonymized Research Export". 3. The backend executes a MongoDB aggregation pipeline. 4. The system formats the results into a downloadable JSON file. 5. The admin downloads the file successfully.
Alternative	If a standard user attempts to call the export API endpoints, the system immediately returns a 403 Unauthorized error and denies access.

Table 6.1: Textual Description: Export System Data

6.3.2. Activity Diagram

Figure 6.2 presents the activity diagram modeling the administrative operations.

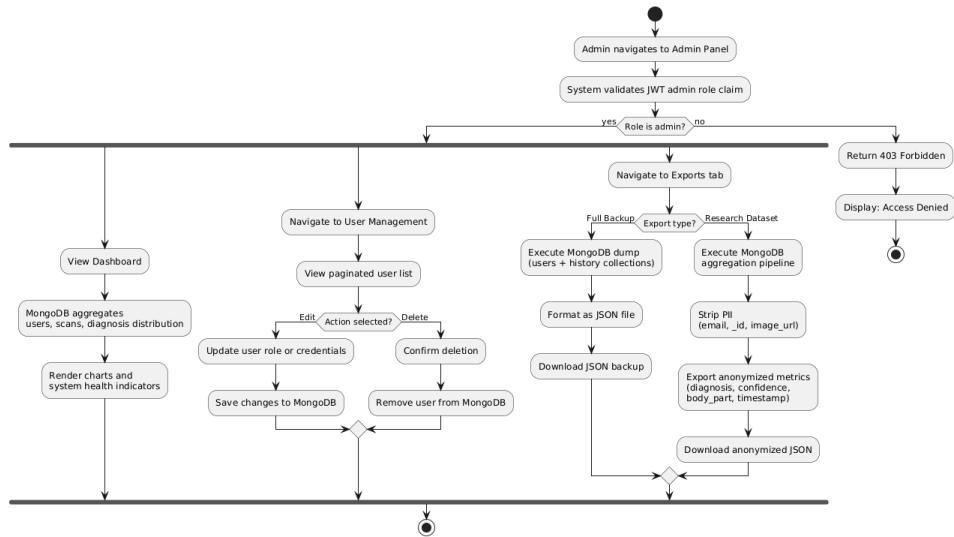


Figure 6.2: Activity Diagram for Sprint 4

6.3.3. Class Diagram

Figure 6.3 shows the conceptual class diagram for the entities managed during this sprint.

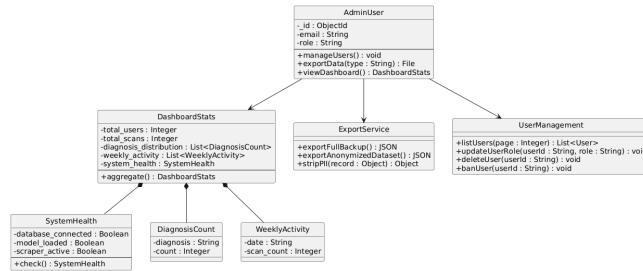


Figure 6.3: Conceptual Class Diagram for Sprint 4

6.3.4. Sequence Diagram

Figure 6.4 details the flow of data when an administrator requests system statistics for the dashboard.

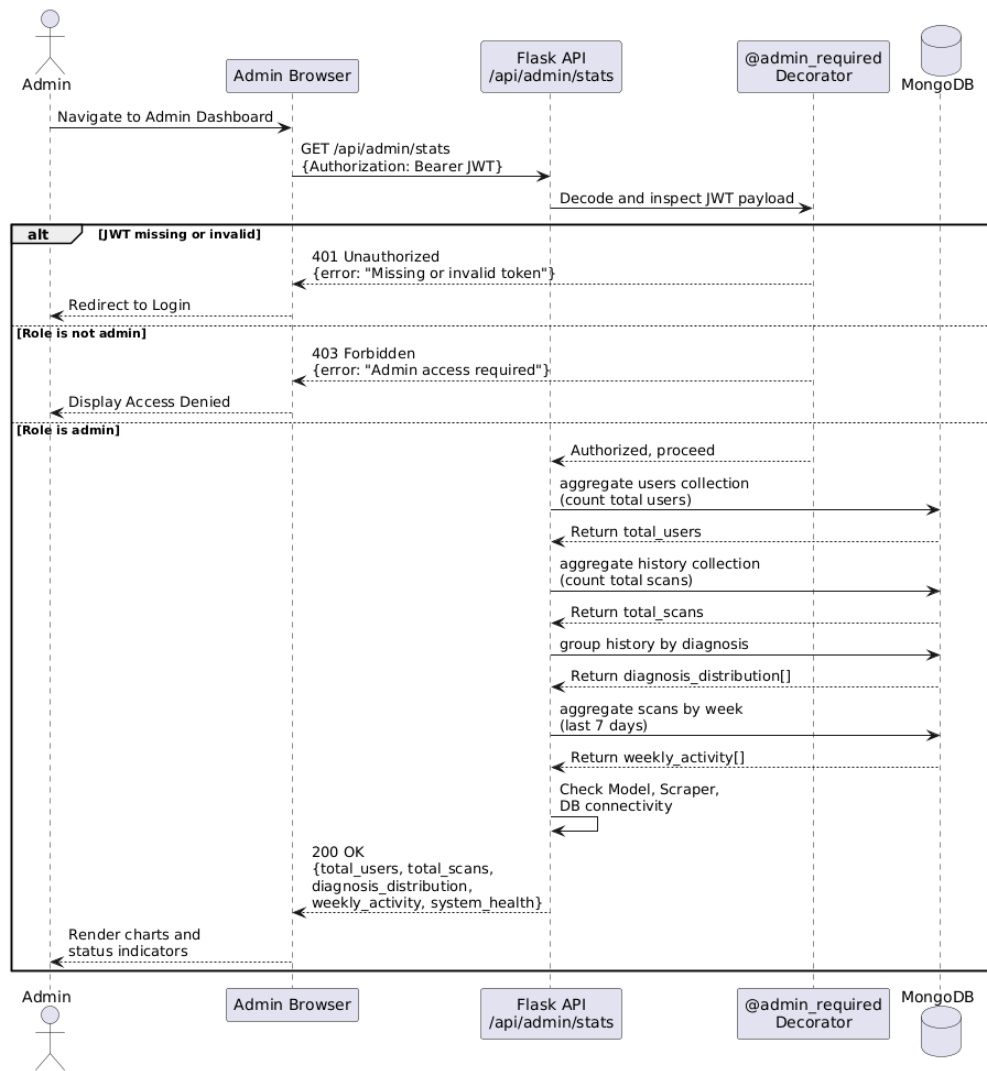


Figure 6.4: Sequence Diagram: Admin Dashboard Visualization

6.4. Tasks

The specific development tasks required to fulfill the administrative objectives are documented in the Sprint 4 backlog.

ID	User Story	Task Description
11	As an admin, I want a centralized dashboard to monitor system activity, usage trends, and platform health.	Develop the <code>/stats</code> backend endpoint utilizing MongoDB aggregations to gather platform metrics.

		Design the React Admin Dashboard overview using charting libraries to visualize data.
12	As an admin, I want to manage registered users.	Develop the User Management UI (View, Edit, Delete modals).
		Implement the secure backend CRUD endpoints with strict admin role validation.
13	As an admin, I want to export the database for backup and dataset for future training.	Implement the full database backup generation logic in Flask.
		Implement the MongoDB projection pipeline to strip PII and export anonymized research data.

Table 6.2: Sprint 4 Backlog

6.5. Development

This section presents the final administrative interfaces and verifies the security constraints applied to the backend API.

6.5.1. User Interfaces

Admin Overview Dashboard

The primary dashboard provides a real-time overview of platform health, showing system statuses (Database, Model, Scraper connectivity), weekly scanning activity charts, and the overall distribution of identified diseases across the platform. Figure 6.5 presents the administrative overview dashboard.

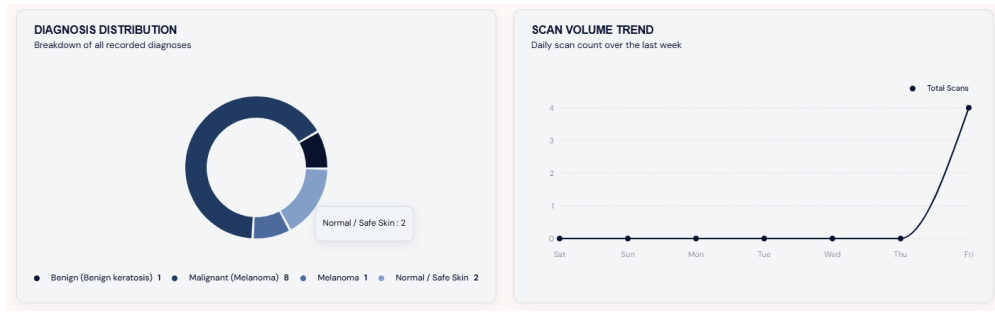


Figure 6.5: Administrative Overview Dashboard

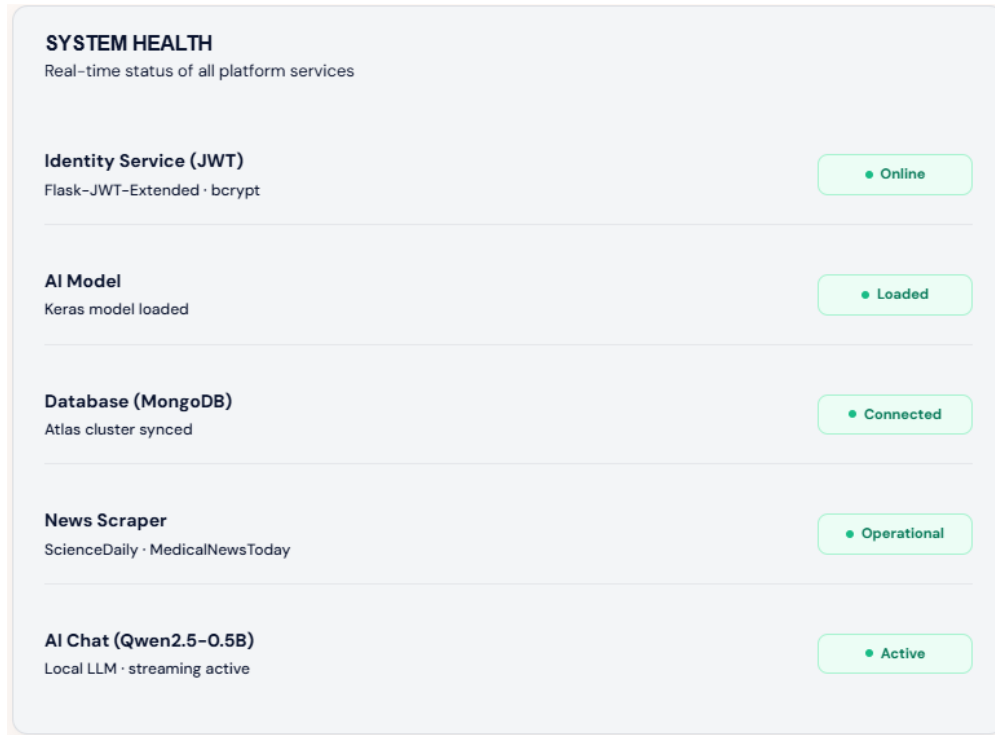


Figure 6.6: System Health and Status Dashboard

User Management and Export Interfaces

The platform allows administrators to perform critical operations via dedicated tabs. The User Management tab features a paginated data table to directly update user credentials or ban accounts. The Exports tab securely generates and downloads the JSON backup files. Figure 6.7 shows the user management and export interface.

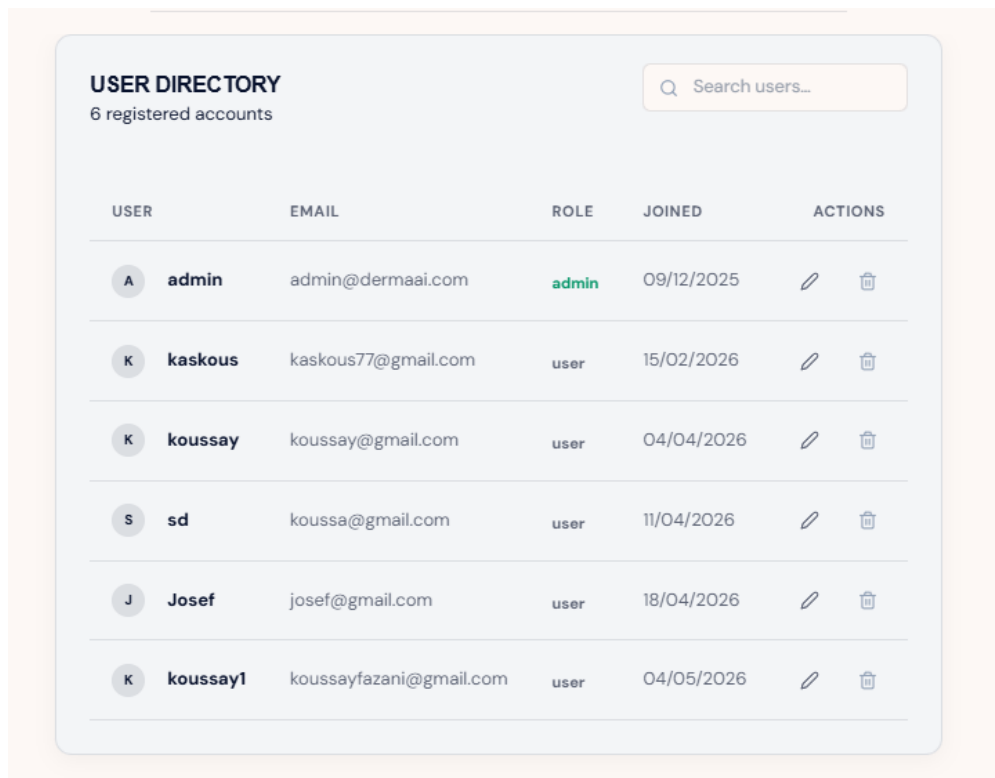


Figure 6.7: User Directory and Role Management Interface



Figure 6.8: Database Export Configuration Interface

News and Blog Interface

The platform also includes a dynamic blog component that fetches and displays the latest dermatological news, creating an engaging content hub for users to stay informed on health trends. Figure 6.9 presents the blog interface.

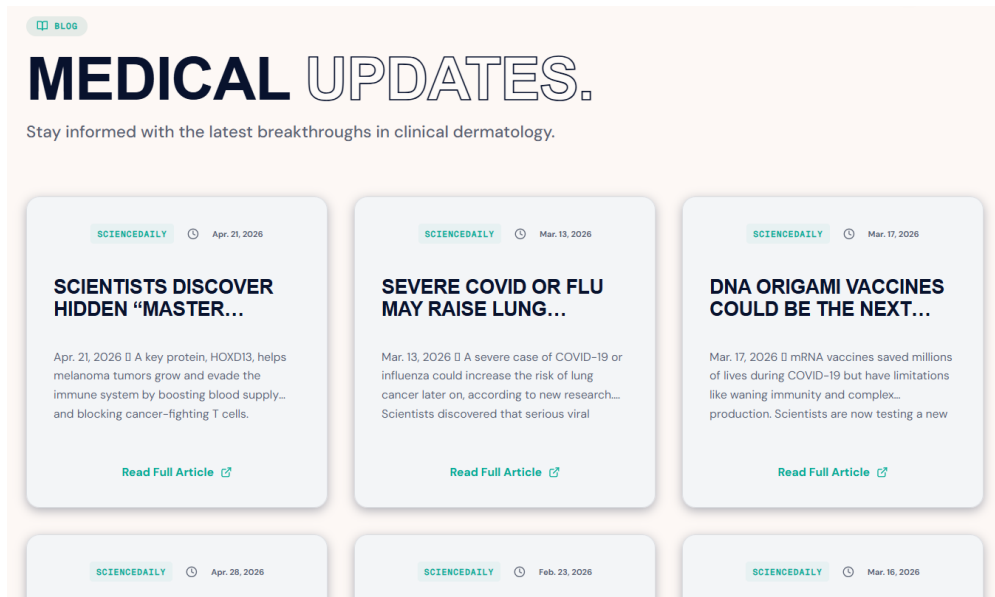


Figure 6.9: Dermatology News and Blog Interface

6.5.2. API Testing & Security Validation

Because administrative functions handle highly sensitive data, testing focused on security. The `@admin_required` Python decorator was verified using Postman. When a standard user (lacking the “admin” role in their JWT payload) attempts to access `/api/admin/users` or `/api/admin/backup`, the server correctly intercepts the request and returns a 403 Forbidden status, ensuring the system cannot be compromised via privilege escalation. Figure 6.10 illustrates this security validation.

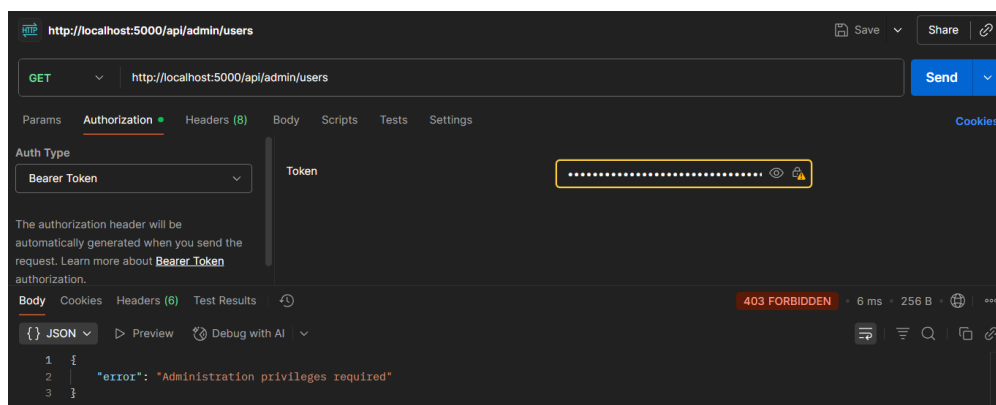


Figure 6.10: Postman Test: Admin Privilege Enforcement

6.6. Administrative Control System

To finalize the platform’s lifecycle management, specific backend architectures were engineered to handle data scaling, privacy, and system security.

6.6.1. Role-Based Access Control (RBAC)

Platform security relies on JWT (JSON Web Token) payload claims. During the authentication phase, the system verifies the user's role against the MongoDB database. If the role is defined as 'admin', this claim is cryptographically signed into the token. The backend Python routes use the `@admin_required` wrapper to actively decode and inspect this signature on every protected request, creating a robust, stateless security barrier around the administrative API.

6.6.2. Data Anonymization Pipeline

The export functionality was split into two distinct mechanisms to serve different operational needs:

- **Full Backup:** Dumps exact, raw JSON representations of the `users` and `history` collections. This is used strictly by administrators for emergency disaster recovery.
- **Research Dataset Export:** Built specifically to fuel future iterations of the deep learning model. The system uses a MongoDB Aggregation Pipeline (`$project`) to automatically strip all Personally Identifiable Information (PII) before the file is generated. It actively removes the `user_email`, `system_id`, and localized `image_url`, exporting only the raw medical metrics (diagnosis, confidence levels, body part, and timestamp). This ensures medical data compliance while providing high-quality retraining data.

6.7. Progress Tracking

Figure 6.11 highlights the Trello Kanban board upon completion of the final sprint, signifying the end of active development for the core DermaAI platform.

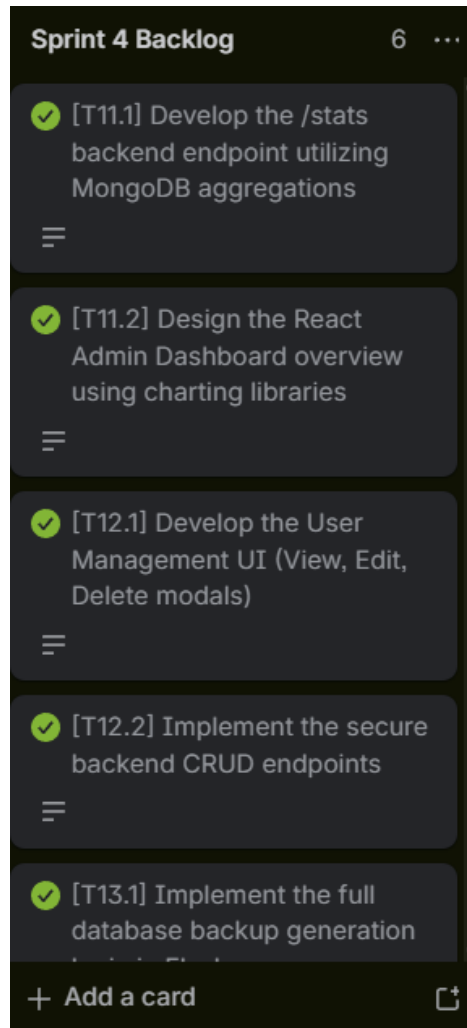


Figure 6.11: Trello Board at the End of Sprint 4

6.8. Conclusion

Sprint 4 successfully established the critical administrative infrastructure required to monitor and secure the DermaAI platform. By providing real-time visual insights, direct user management, and anonymized data export capabilities, the system is fully prepared for production deployment. This marks the end of the development phase, resulting in a cohesive and secure medical application.

GENERAL CONCLUSION

The DermaAI platform project represents a significant step toward modernizing Tunisian healthcare by leveraging the power of technology to address the sector's challenges. Through the integration of artificial intelligence, web development, and localized conversational agents, the platform empowers users with tools to perform preliminary skin lesion assessments, understand their results through visual explainability, and access continuous medical guidance. The development process, from design to implementation, has been a journey of learning and innovation, resulting in a solution that meets the needs of its target users while prioritizing data security and clinical accuracy.

The evaluation phase confirmed the platform's effectiveness, with strong diagnostic performance metrics, robust API interactions, and a highly user-friendly interface. Despite challenges such as deep learning integration and medical data privacy, the solutions implemented—such as modular architectures and localized offline language models—ensured the platform's success. Looking ahead, the proposed future enhancements, including a mobile app, direct teledermatology integration with clinics, and expanded multilingual support, will further expand the platform's impact, making it a cornerstone of healthcare innovation in Tunisia.

In conclusion, the DermaAI platform project not only fulfills the objectives of our Final Year Project but also contributes to the broader goal of digital transformation in Tunisian healthcare. We hope that this work inspires further advancements in the field, paving the way for a more accessible, efficient, and proactive future for patients across the country.

BIBLIOGRAPHY

- [1] **Arnold, M. et al.** Global Burden of Cutaneous Melanoma in 2020 and Projections to 2040. *JAMA Dermatology*, vol. 158, no. 5, pp. 495–503, 2022.
- [2] **Liu, C. et al.** Global Burden of Cutaneous Malignant Melanoma from 1990 to 2021 and Prediction to 2045. *Frontiers in Oncology*, vol. 14, 2024.
- [3] **Skin Cancer Foundation.** Skin Cancer Facts & Statistics [Online]. Available at:
<https://www.skincancer.org>, 2024.
- [4] **Wu, Y. et al.** Skin Cancer Classification with Deep Learning: A Systematic Review. *Frontiers in Oncology*, vol. 12, 2022.
- [5] **Kassem, M. A. et al.** Machine Learning and Deep Learning Methods for Skin Lesion Classification. *Diagnostics*, vol. 11, no. 8, 2021.
- [6] **Debelee, T. G.** Skin Lesion Classification and Detection Using Machine Learning: A Systematic Review. *Diagnostics*, vol. 13, no. 19, 2023.
- [7] **Abdalla, A. R. et al.** Deep Learning for Detection of Suspicious Pigmented Skin Lesions: A Systematic Review. *Cureus*, vol. 16, no. 7, 2024.
- [8] **Journal of the American Academy of Dermatology.** Global Skin Cancer Incidence and Mortality with a Focus on Dermatologist Density. 2024.
- [9] **SkinVision B.V.** SkinVision App [Online]. Available at:
<https://www.skinvision.com>, 2024.
- [10] **iDoc24 Inc.** First Derm App [Online]. Available at:
<https://www.firstderm.com>, 2024.
- [11] **Schwaber, K. and Sutherland, J.** The Scrum Guide [Online]. Available at:
<https://scrumguides.org>, 2020.

- [12] **Schwaber, K. and Sutherland, J.** The Scrum Guide. [Online]. Available at: <https://scrumguides.org>, 2020.
- [13] **Selvaraju, R. R. et al.** Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *International Journal of Computer Vision*, vol. 128, pp. 336–359, 2020.
- [14] **Tschandl, P. et al.** The HAM10000 Dataset: A Large Collection of Multi-Source Dermatoscopic Images of Common Pigmented Skin Lesions. *Scientific Data*, vol. 5, no. 180161, 2018.
- [15] **Chollet, F.** Xception: Deep Learning with Depthwise Separable Convolutions. *Proceedings of IEEE CVPR*, pp. 1251–1258, 2017.
- [16] **Lin, T.-Y. et al.** Focal Loss for Dense Object Detection. *Proceedings of IEEE ICCV*, pp. 2980–2988, 2017.
- [17] **Bai, J. et al.** Qwen Technical Report [Online]. *arXiv preprint*, arXiv:2309.16609, 2023. Available at: <https://arxiv.org/abs/2309.16609>.
- [18] **Ronacher, A.** Flask: A Lightweight WSGI Web Application Framework [Online]. Available at: <https://flask.palletsprojects.com>, 2024.
- [19] **Meta Open Source.** React: A JavaScript Library for Building User Interfaces [Online]. Available at: <https://react.dev>, 2024.
- [20] **MongoDB, Inc.** MongoDB Documentation [Online]. Available at: <https://www.mongodb.com/docs>, 2024.
- [21] **Abadi, M. et al.** TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems [Online]. Available at: <https://www.tensorflow.org>, 2015.
- [22] **Jones, M. et al.** JSON Web Token (JWT). *RFC 7519*, Internet Engineering Task Force (IETF), 2015.

- [23] **Richardson, L.** Beautiful Soup Documentation [**Online**]. Available at:
<https://www.crummy.com/software/BeautifulSoup>, 2024.
- [24] **Andi, P. et al.** jsPDF: Client-side JavaScript PDF Generation [**Online**]. Available at:
<https://github.com/parallax/jsPDF>, 2024.
- [25] **OWASP Foundation.** OWASP Top Ten 2021 [**Online**]. Available at:
<https://owasp.org/Top10>, 2021.

AI-Based Skin Analysis AND Lesion Detection

Fezzani Koussay

الخلاصة : تمت كتابة هذه الرسالة كجزء من مشروع نهاية الدراسات للحصول على الإجازة في علوم الكمبيوتر والوسائط المتعددة بـ ISIMS. يهدف المشروع لتطوير منصة الويب **DermaAI** للكشف المبكر عن الآفات الجلدية باستخدام نمذجة **UML**. تم تطوير الواجهة الخلفية باستخدام **Python** و **Flask** مع قاعدة بيانات **MongoDB**، والواجهة الأمامية باستخدام **React** و **Vite**. يعتمد نظام الذكاء الاصطناعي على نموذج التعلم العميق **Xception** مع تقنية **GradCAM** للتفسير البصري، ومساعد طبي ذكي يعتمد على **Google Gemini**.

Résumé : Ce mémoire a été rédigé pour l'obtention de la licence en informatique et multimédia à l'ISIMS. Il présente le développement de **DermaAI**, une plateforme web de détection précoce des lésions cutanées conçue avec **UML**. L'application repose sur un backend **Flask (Python)** couplé à **MongoDB**, et un frontend **React (Vite)**. Le moteur d'intelligence artificielle intègre le modèle de Deep Learning **Xception** avec **Grad-CAM** pour l'explicabilité visuelle des prédictions, ainsi qu'un chatbot médical intelligent basé sur l'API **Google Gemini**.

Abstract: This thesis was written to obtain the bachelor's degree in computer science and multimedia at ISIMS. It presents the development of **DermaAI**, a web platform for early skin lesion detection designed using **UML**. The application features a **Flask (Python)** backend with **MongoDB**, and a **React (Vite)** frontend. The AI engine combines an **Xception**-based Deep Learning model with **Grad-CAM** for visual explainability, alongside an interactive medical chatbot powered by the **Google Gemini** API.

المفاتيح : منصة ويب، كشف الآفات الجلدية، التعلم العميق، روبوت دردشة ذكي

Mots clefs: Plateforme web, Détection de lésions, Deep Learning, Xception, Flask, React, MongoDB, UML, Chatbot IA.

Key-words: Web platform, Skin lesion detection, Deep Learning, Xception, Flask, React, MongoDB, UML, AI Chatbot

